

Body-decoupled grounding via reduction: A novel approach on the ASP bottleneck

Viktor Besin ^a, Markus Hecher ^{b,*}, Matthias König ^a, Stefan Woltran ^a

^a TU Wien, Institute of Logic and Computation, Favoritenstraße 9–11 / E192, Vienna, 1040, Vienna, Austria

^b University of Artois, UMR8188, CNRS, CRIL, Rue Jean Souvraz, SP 18, Lens, 62307, France

ARTICLE INFO

2020 MSC:

68N17

68T30

68T27

Keywords:

Logic programming

ASP

Grounding bottleneck

Solver

ABSTRACT

Answer Set Programming (ASP) is used in many real-world applications as an easy-to-use and efficient modeling paradigm. In ASP, rules (containing variables) are instantiated against facts to derive conclusions (i.e., grounding the program). However, a large number of variables and facts (more precisely: domain values) leads to the well-known “grounding bottleneck”: the variables in a rule are instantiated by every combination of domain values, leading to potentially infeasibly large ground programs. This is especially problematic for rules with large rule bodies containing many variables, as standard grounding approaches cause an exponential blowup in the total number of variables in a rule. This paper presents a novel method to overcome this obstacle. In contrast to the standard approach, our technique “decouples” the body atoms, i.e., the instantiation handles each body atom separately. In particular, our procedure translates a non-ground tight program to a ground disjunctive program that is exponential only in the maximum predicate arity, and is thus polynomial if this arity is fixed to a constant. We extend this to non-ground, normal programs, and ultimately use similar ideas to translate non-ground, disjunctive programs to ground epistemic logic programs—all while decoupling the body atoms. We demonstrate the feasibility of this new method experimentally by comparing it to standard ASP technology in terms of grounding size, grounding time and total runtime.

1. Introduction

Answer Set Programming (ASP) [1–4] is a well-known problem modeling and solving framework, which is, together with other formalisms (see e.g., QSAT [5,6], #SAT [7]), known to be the main driver for practical realisations in the fields knowledge representation and reasoning (KR), as well as symbolic artificial intelligence. ASP can be seen as an extension of propositional satisfiability (SAT), where knowledge is expressed by means of rules comprising a (*logic*) *program*, whose solutions are sets of atoms, called *answer sets*, that obey every rule. Similar to SAT, the development of efficient ASP solvers [2,8] enables the broad use not only for academic purposes, e.g., in natural language understanding [9], but also in industrial applications [10]. Beyond that, there are even computationally harder extensions to ASP, e.g., the epistemic extension *Epistemic Logic Programming* (ELP) [11–14], which allows for reasoning over multiple worlds and has recently gained popularity among researchers. This makes the improvement of ASP systems an important and interesting research topic.

* Corresponding author.

E-mail addresses: viktor.besin@tuwien.ac.at (V. Besin), hecher@cril.fr (M. Hecher), matthias.koenig@tuwien.ac.at (M. König), stefan.woltran@tuwien.ac.at (S. Woltran).

<https://doi.org/10.1016/j.artint.2026.104600>

Received 10 November 2024; Received in revised form 22 May 2026; Accepted 6 August 2026

Available online 11 August 2026

0004-3702/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Table 1

Known complexity results for the consistency (for entities marked with an asterisk: inconsistency) of the program types discussed in this work. The proposed reductions are highlighted with arrows.

	Ground	Non-Ground (bounded arity)
Horn Programs*	P	NP
Tight/Normal Programs	NP	Σ_2^P -c
Disjunctive Programs	Σ_2^P -c	Σ_3^P -c
Epistemic Programs	Σ_3^P -c	-

A particular strength of ASP as a KR language is its first-order variant where rules are given over variables. Such *non-ground logic programs* require grounding (i.e., the *instantiation* of variables necessary for solving) before calling an ASP solver. Traditional ASP systems therefore rely on a so-called *ground-and-solve* technique [15], where a grounding module transforms the non-ground input program into its propositional counterpart by replacing variables with constants and a solving module computes the answer sets accordingly. Ground-and-solve approaches also reach other areas of artificial intelligence, like planning [16,17]. There, besides trying to avoid full grounding via lifted planning [18,19], improvements inspired by the work of this manuscript have been proposed [20] and implemented in a winning solver at the international planning competition 2023. Note that grounding is not restricted to specific formalisms, and has been also used for, e.g., efficient model counting [21]. This approach, however, has its limitations for certain problem types, which applies especially for problems where the grounding leads to a combinatorial blow up, i.e., the instantiation of rules yields exponentially large programs whose evaluation is not processable by the solver. This problem is usually referred to as *grounding bottleneck* of ASP (cf. [Example 1.1](#)).

Example 1.1. ASP can be used to encode many problems effortlessly. Assume the problem of choosing relationships among entities such that the structure does not contain a triangle. This can easily be represented as a graph consisting of vertices and edges, and encoded in the following (non-ground) program.

Program Π decides in Rule (1) for each edge (predicate e) of a given graph, whether to pick it (predicate p) or not (predicate $\bar{p}$). Then, in Rule (2) it is ensured that the choice of edges does not form triangles.

$$p(a, b) \vee \bar{p}(a, b) \leftarrow e(a, b). \quad (1)$$

$$\leftarrow p(x, y), p(y, z), p(x, z), x \neq y, y \neq z, x \neq z. \quad (2)$$

Let us inspect variables x , y , and z when instantiated during grounding. Traditional grounding generates a ground rule for every possible variable allocation from the program's domain $D = \text{dom}(\Pi)$, which are constant symbols over a set $F(\Pi)$ of facts. Here these facts comprise a graph G provided via a set of expression of the form $e(a, b)$, $e(b, c)$, etc.; in other words, $|D|$ is the number of vertices in G . Therefore, the grounding effort is

in $\mathcal{O}(|D|^3)$. In general, the effort of grounding a given program Π is $\mathcal{O}(|\text{dom}(\Pi)|^n)$ where n is the largest number of distinct variables in a rule.

To call attention to our novel grounding approach, we recall known ASP complexity results. We identify different program types for logic programs, where the complexity of these types typically ascend with their expressiveness and structure. As seen in [Table 1](#), the computational complexity for *ground* programs is relatively mild: deciding whether a *disjunctive* program has an answer set is located at the second level of the polynomial hierarchy [22], whereas *normal* (and *tight*) programs yield NP-complete fragments [23,24].

While variables in *non-ground* programs increase a program's expressiveness and reduce the encoding effort, the complexity reflects the potentially huge costs of grounding, which is why non-ground programs ascend up to NEXPTIME-completeness [25]. However, earlier works (see [26]), which focused on programs in bounded settings, show that the complexity of programs with bounded predicate arity is more narrow. In particular, the results show that the complexity for non-ground, normal (and tight) programs drop to Σ_2^P completeness and for non-ground, disjunctive programs to Σ_3^P completeness (cf. [Table 1](#)). What is even more interesting: the exponential blow up in rule instantiation *still holds* for bounded predicate arity despite their milder complexity.

Resting upon these results, we can identify that non-ground programs (under bounded arity setting) and ground programs share the same complexity in certain cases (cf. [Table 1](#)), which allows us to utilize reductions as an efficient alternative solving approach. In particular, this gives rise to a polynomial reduction between non-ground and ground logic programs and thus an *alternative grounding procedure that delegates certain efforts to the solving process*. In the first place, this idea seems qualified for a reduction from non-ground, normal (and tight) to ground, disjunctive programs (cf. [Table 1](#) ①). But, as the development of competitive epistemic solvers [27–29] suggests, there might even lie potential in a reduction from non-ground, disjunctive ASP programs to ground ELP programs (cf. [Table 1](#) ②), which are known to be Σ_3^P -complete as well [13].

While the reduction itself seems interesting enough, the encoding of such a reduction can be constructed in such a way that the dependencies of predicates in rule bodies are decoupled, i.e., each predicate in the rule's body can be instantiated separately to reduce

the number of instantiations. Especially, when assuming large rule bodies with very dense structures, this idea might be particularly useful. The difference of this concept, which we call *body-decoupled grounding*, compared to traditional grounding systems is shown in the following example.

Example 1.2. Recall the program Π of [Example 1.1](#). As seen above, traditional grounding systems instantiate every variable of a rule at once yielding a grounding effort of $\mathcal{O}(d^3)$, $\mathcal{O}(|\text{dom}(\Pi)|^n)$ in general.

Our approach is different, since it decouples rule bodies during the grounding of Π . Therefore, body predicates of (2) are grounded individually, yielding ground programs that are linear in the size of the ground atoms. Here, it corresponds to $\mathcal{O}(|\text{dom}(\Pi)|^2)$ due to arity 2. In general, the effort yields $\mathcal{O}(|\text{dom}(\Pi)|^c)$ where c is a fixed constant corresponding to the predicate arity.

Contributions

By utilizing the aforementioned complexity results we provide an alternative grounding approach that decouples rule bodies during grounding, which is rectified during solving.

So, we address grounding from a different perspective: Can we translate non-ground programs to ground programs without an explicit instantiation of rules, but where instead certain aspects of grounding are delegated to the efficient search procedures of modern ASP/ELP solvers? And, is this a promising approach, at least, for certain program classes?

Our main contributions are summarized as follows.

1. We present a novel reduction from non-ground, tight logic programs to ground, disjunctive programs (cf. [Table 1](#) ①) that encodes grounding via search by identifying unsatisfied ground rules and unfounded atoms.
In contrast to traditional grounding, our reduction allows us to *decouple* predicates occurring in the body of a rule, which might be particularly useful for larger bodies with a very dense structure.
We show that in general the jump from (non-ground) tight programs to (ground) disjunctive programs cannot be avoided.
2. We extend this approach to non-ground, normal programs, where for ensuring foundedness we additionally encode the idea of orderings (level mappings) in our reduction. Based on these encodings we introduce an even computationally cheaper reduction for non-ground, Horn programs, that is capable of deciding inconsistency and membership of atoms by translating to ground, normal programs.
3. We lift our idea to a reduction to epistemic logic programs. Due to the increased complexity of these programs we can extend this approach to non-ground, disjunctive programs (cf. [Table 1](#) ②). The presented reduction encodes subset-minimization of answer sets and can therefore omit foundedness and orderings.
4. We present a prototype that allows to translate critical parts of tight programs using our reduction, thereby empowering the grounding process by decoupling body predicates. Preliminary experiments indicate that this approach can lead to significant speed-ups and a massive reduction in grounding size compared to state-of-the-art grounders.

This is an extended version of papers [\[30,31\]](#) that appeared in Proceedings of the 31st International Joint Conference on Artificial Intelligence (IJCAI 22) and of the 26th European Conference on Artificial Intelligence (ECAI 23). In addition to the conference versions, this work contains additional examples and full proofs. Further, we added contributions by encoding a translation from non-ground, Horn programs.

2. Related work

The roots of Answer Set Programming (ASP) [\[32\]](#) can be traced back to Prolog, but the most fundamental work was achieved by defining stable model semantics [\[33\]](#), which then led to answer sets [\[34\]](#). The theoretical logic-based foundation is the Logic of Here-and-There [\[35\]](#), which has led to years of fruitful research [\[36\]](#). Overviews of these developments are provided by seminal works [\[1,3,4,37\]](#).

In fact, the practical success of ASP can be attributed to the early availability of grounders and solvers. The SMODELS system [\[38\]](#) was one of the earliest solvers, followed by the DLV system [\[39\]](#). There are also SAT-based solvers such as CMODELS [\[40\]](#) and ASSAT [\[41\]](#), as well as newer developments [\[42\]](#). One of the most important developments was the development of conflict-driven answer set solving systems [\[43\]](#), such as CLASP [\[44\]](#) and WASP [\[45\]](#).

Grounding [\[46\]](#) is the instantiation of variables with domain values, which is achieved by grounders. Most modern solvers have a grounder attached to them. Efficient grounding is an active field of research and requires powerful rule initialization procedures trying to evade the intractable evaluation of non-ground programs. Historically, for finite groundings, programs were forced to comply with ω - or λ -restricted programs [\[47,48\]](#); the latter work also introduced the first version of GRINGO. State-of-the-art grounders, like the latest version of GRINGO [\[49\]](#), the grounder of DLV [\[50\]](#), and IDLV [\[51\]](#), use more involved instantiation techniques such as semi-naïve database evaluation.

Nowadays, grounders and solvers are sometimes combined into one coherent systems like CLINGO [\[52\]](#). Typically they work independently, where a program is first grounded and then solved, which gives rise to the ground-and-solve paradigm. Multi-shot solving reduces the necessary amount of grounding via repeated grounding calls [\[2\]](#). A more in-depth overview is provided in the literature [\[15\]](#). As the grounding step is highly exponential (in the worst case), and the evaluation of non-ground ASP programs reaches NEXPTIME [\[25\]](#) and beyond [\[53\]](#), large groundings may prohibit the evaluation of programs. This problem is called the *grounding bottleneck* [\[54,55\]](#).

Comparison to Existing Grounding Techniques. The literature distinguishes many approaches, ranging from traditional instantiation tactics [2,46,56] over fruitful estimations [57] and lazy grounding [58,59]. Besides, there are further attempts to avoid the grounding bottleneck, like constraint-programming or ASP modulo theory extensions, e.g., [60–62], and methods based on graph invariants like treewidth [8,63–65]. Below, we will elaborate and relate to these approaches in more detail.

Lazy grounding combines grounding and solving into a conceptually single step. The goal is to prevent unnecessary instantiations of rules, which is achieved by instantiating rules only if their body is satisfied. Although the technique is promising, current lazy systems are overall not competitive with ground-and-solve systems [54]. Prominent traditional lazy-grounding approaches are GASP [66], ASPERIX [67], and OMIGA [68], which rely on sequences of firing rules (different from the atom-focused CDCL approaches). The lazy-grounding system of ALPHA [69] adapts the CDCL algorithm to combine benefits of lazy grounding with CDCL. Other research extends the input language of lazy solvers to include aggregates [58]. Some techniques use lazy grounding with specialized search techniques [70]. Further work introduces prominent solver techniques into lazy grounding, such as domain-independent heuristics for speeding up the procedure [59,71]. Note that in contrast to lazy grounding, our approach is not lazy but can be seen as exact, as our grounding effort for each predicate is still fully carried out in a dedicated grounding phase.

An approach related to lazy grounding is the *compilation-based* technique. Here parts of a program are typically grounded traditionally and serve as domains, whereas other parts are compiled into efficient subprocedures. These subprocedures take over both grounding and solving, injecting additional constraints or nogoods to ensure semantics. Earlier approaches [72] iteratively extend a partial answer set of a program by a compiled process that injects constraints. This process uses compiler techniques such as loop unrolling. A similar approach is DUALGROUNDER [73], which instead of compiling rules to optimized native-code procedures exploits CLINGO's multi-shot capabilities. The other prominent branch of compilation techniques uses extensions of the CDCL procedure (propagators), first demonstrated to be feasible by [74]. Later versions extend this to constraints [9] and tight programs [75], and even aggregates [76]. We refer to [36,77–80] for further readings and different applications in this context. Our approach does not compile grounding effort into code or subprocedures. Instead, we still use grounding, but carry out some aspects of it during solving, thereby still using the vanilla ASP pipeline.

ASP Modulo Theory (ASP^{MT}) is another approach to circumvent the grounding bottleneck. It integrates methods and tools from other paradigms such as constraint-programming [43]. Notable ASP solvers enhanced with CSP are CLINGCON [60,81], EZCSP [82], and EZSMT [83]. Some systems combine ASP with mixed-integer programming, e.g., MINGO [84]. Additional developments include HEX-programs for integrating external tools [85] and bound-founded ASP [86]. These approaches often extend the ASP input language with new constructs and are mostly implemented by extending the CDCL loop of modern solvers [36]. Note that our approach does not rely on theory reasoning at all, which is why we see it as a vanilla ASP approach that is based on exact grounding (despite using auxiliary predicates).

In contrast, our approach can be seen as a complexity-theoretic grounding approach [30,87]. Its foundation lies in the complexity-theoretic discussions of ASP [25,26], showing that ground disjunctive ASP is Σ_2^P -complete, and under bounded predicate arities, non-ground non-disjunctive ASP is also Σ_2^P -complete. Thus, Besin et al. [30] introduced the *Body-Decoupled Grounding* reduction from non-ground non-disjunctive to ground disjunctive ASP. This work also showed excellent practical results on instances exhibiting the grounding bottleneck. Indeed, in this work we also utilize the complexity increase of solving non-ground programs of bounded predicate arity, to only ground predicates but not every predicate combination. This is possible due to the additional level and by the observation that deciding rule satisfiability or atom unfoundedness does not need the whole rule body (at once), but only a single rule body predicate. Later work [87] added aggregates and extended the reduction to a hybrid grounding procedure where programs are manually split between body-decoupled grounded and traditionally grounded parts. Related approaches extend the input language to disjunctive ASP with reductions to epistemic logic programming [28]. Further research leverages *treewidth* to improve splitting and optimization [63,88].

Finally, *meta-programming* in ASP [36] is another important direction. In meta-programming, programs are inputs to other programs that perform computation on them. This concept has been used for extending semantics [79], preference handling (ASPRIN), planning (PLASP [77]), and automatic integration of guess-and-check programs [89]. Similar to our work, Eiter et al. [90] draws on the complexity of bounded arities to design space-efficient ASP evaluation methods, but their method is more in the spirit of meta-programming than on an alternative grounding procedure. Instead, we focus on decoupling body predicates and a translation to ground ASP based on computational complexity.

3. Preliminaries

Before we discuss our reductions, we briefly recall some basics. We use mathematical vectors $X = \langle x_1, \dots, x_m \rangle$, $Y = \langle y_1, \dots, y_n \rangle$ in the usual way; we *combine vectors* by $\langle X, Y \rangle := \langle x_1, \dots, x_m, y_1, \dots, y_n \rangle$ and test whether x_1 is *contained in* X by $x_1 \in X$. Further, we let $Y \setminus X = \langle y_{i_1}, \dots, y_{i_k} \rangle$ be precisely those elements in Y (in the order of appearance) that are not in X , i.e., $\{y_{i_1}, \dots, y_{i_k}\} = \{y \in Y \mid y \notin X\}$. Without loss of generality, we assume that elements of vectors are given in any fixed total order; for a given set S , we *construct* its unique vector by $\langle S \rangle$. Further, we assume familiarity with SAT [5], and follow standard definitions of ASP [1].

3.1. Ground answer-set programming (ASP)

Let $\ell, m, n \geq 0$ be non-negative integers such that $\ell \leq m \leq n$; $a_1, \dots, a_n$ be distinct propositional atoms. We refer to a *propositional atom* or the negation of it by *literal*. A ground (*disjunctive*) *program* P is a set of (*disjunctive*) rules of the form

$$a_1 \vee \dots \vee a_\ell \leftarrow a_{\ell+1}, \dots, a_m, \neg a_{m+1}, \dots, \neg a_n. \quad (3)$$

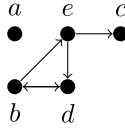
Fig. 1. Dependency graph D_{P_1} of P_1 (cf. Example 3.1).

Table 2

Decision problems related to the answer-set programming formalism that are discussed in the course of this work. The column “Fragment” states the corresponding fragment of logic programs, while the columns “Ground” and “Non-Ground (bounded arity)” list the respective complexity (e.g., “NP” means the problem is NP-complete).

Decision Problem	Fragment	Ground	Non-Ground (bounded arity)
TIGHT ASP	tight programs	NP [24]	Σ_2^P [26]
NORMAL ASP	normal programs	NP [23]	Σ_2^P [26]
HORN ASP	Horn programs	P [92]	co-NP [92]
DISJUNCTIVE ASP	disjunctive programs	Σ_2^P [22]	Σ_3^P [26]
ELP	epistemic programs	Σ_3^P [13]	–

For a rule r , we let $H_r := \{a_1, \dots, a_\ell\}$ be the *head* of r , $B_r^+ := \{a_{\ell+1}, \dots, a_m\}$ be the *positive body* and $B_r^- := \{a_{m+1}, \dots, a_n\}$ be the *negative body* of r . We refer by $B_r^+ \cup B_r^-$ to the *body* of r . We denote the sets of atoms occurring in a rule r or in a program P by $\text{at}(r) := H_r \cup B_r^+ \cup B_r^-$ and $\text{at}(P) := \bigcup_{r \in P} \text{at}(r)$. Further, a rule r is *normal* if $|H_r| \leq 1$ and a program Π is *normal* if all its rules are normal, otherwise the program is called *disjunctive*. We call a program *Horn*, if all its rules are normal and positive, i.e., $|B_r^-| = 0$. Note that this *includes rules with empty rule heads* ($H_r = \emptyset$), i.e., we allow a rule that is sometimes referred to as constraint. Further, we can identify *tight* programs by the following definition.

Definition 3.1 (Dependency Graph). The *dependency graph* D_P of P is the directed graph defined on the set $\bigcup_{r \in \Pi} H_r \cup B_r^+$ of atoms, where for every rule $r \in P$ two atoms $a \in B_r^+$ and $b \in H_r$ are joined by an edge (a, b) . A program P is called *tight* if D_Π has no directed cycle [91].

Example 3.1. Consider the program

$$P_1 := \{ \overbrace{a \vee b \leftarrow .}^{r_1}, \overbrace{c \vee d \leftarrow e.}^{r_2}, \overbrace{e \leftarrow b, \neg d.}^{r_3}, \overbrace{e \leftarrow \neg b.}^{r_4}, \overbrace{d \leftarrow b, \neg e.}^{r_5}, \overbrace{b \leftarrow d, \neg e.}^{r_6} \}.$$

We can observe that the program P_1 is not tight, since the dependency graph D_{P_1} , seen in Fig. 1, contains the directed cycle (b, d, e) . Further, since we have rules r_1 and r_2 , where we have $|H_r| > 1$ for $r \in \{r_1, r_2\}$, the program is neither normal nor Horn, but it is disjunctive

Definition 3.2. (Interpretation) An *interpretation* I is a set of atoms. A rule r is *satisfied* by an interpretation I if $(H_r \cup B_r^-) \cap I \neq \emptyset$ or $B_r^+ \setminus I \neq \emptyset$. An interpretation I is a *model* of a program P if it satisfies every rule $r \in P$, in symbols $I \models P$. The Gelfond-Lifschitz (GL) reduct of P under I is the program P^I obtained from P by first removing all rules r with $B_r^- \cap I \neq \emptyset$ and then removing all $\neg z$ where $z \in B_r^-$ from the remaining rules [11]. An interpretation I is an *answer set* of a program P , if I is a minimal model (w.r.t. $\subseteq$) of P^I , i.e., I satisfies every rule r of the GL reduct P^I and every proper subset of I does not satisfy P^I . We refer to the set of answer sets of a given program P by $AS(P)$.

Example 3.2. Recall the program P_1 of Example 3.1. We can show that the set $\{a, c, e\}$ is an answer set of P_1 , since it is a minimal model of the GL reduct $P_1^{[a, c, e]} = \{a \vee b \leftarrow ., c \vee d \leftarrow e., e \leftarrow b., e \leftarrow .\}$. Furthermore, we can prove the set $\{b, d\}$ as an answer set, since it is a minimal model of $P_1^{[b, d]} = \{a \vee b \leftarrow ., c \vee d \leftarrow e., d \leftarrow b., b \leftarrow d.\}$. Similarly, one can find all answer sets of the program, which are $AS(P_1) = \{\{a, c, e\}, \{a, d, e\}, \{b, c, e\}, \{b, d\}\}$.

Proposition 3.1. The decision problem of deciding whether an ASP program has an answer set, which is referred to as *consistency*¹, is (in general) Σ_2^P -complete [22]. If the input is restricted to normal or tight programs, the complexity drops to NP-completeness [23,24], as seen in Table 2.

The following characterization of answer sets is often applied for normal programs and based on orderings (level mappings) [93,94].

Definition 3.3. (Answer Sets) Let I be a model of a normal program P and φ be a function (ordering) $\varphi : I \rightarrow \{0, \dots, |I| - 1\}$ over I . We say that a rule $r \in P$ is *suitable for founding* $a \in I$ if (i) $a \in H_r$, (ii) $B_r^+ \subseteq I$, (iii) $I \cap B_r^- = \emptyset$, as well as (iv) $I \cap (H_r \setminus \{a\}) = \emptyset$. An atom $a \in I$ is *founded* if there is a rule $r \in P$ founding a , which is the case if r is suitable for founding a and $\varphi(b) < \varphi(a)$ for every $b \in B_r^+$, and *unfounded*² otherwise.

¹ Sometimes the consistency of an answer set is called coherence. However, here we stick to terms consistency and its inverse inconsistency.

² Note that here we do not need so-called unfounded sets which have a reserved meaning for logic programming [95,96].

Then, I is an answer set of P if (i) I is a model of P , and (ii) I is founded, i.e., every $a \in I$ is founded.
For tight programs, the ordering φ is not needed.

Note that this concept goes beyond supported models [97] and indeed captures answer sets, as briefly discussed below. Crucially, for every answer set I , we require a single ordering φ .

Example 3.3. Recall the program P_1 of Example 3.1.

As shown in Example 3.2, the set $I := \{a, c, e\}$ is an answer set of P_1 . This can also be proven by $I \models P_1$ and an ordering $\varphi := \{a \mapsto 0, e \mapsto 1, c \mapsto 2\}$. Then, the atom a is founded by rule r_1 , atom e by rule r_4 and atom c by rule r_2 .

3.1.1. The saturation technique for ASP

As already discussed above, deciding the consistency of a ground disjunctive logic program is Σ_2^P -complete [22]. In the hardness proof, the so-called *saturation technique* has been established³, which became pretty standard, see, e.g., [80,98,99]. This technique is named after its purpose: In order to model the second level of the polynomial hierarchy (with an intuitive, additional universal quantification), the answer set will be saturated by deriving all atoms subject to universal quantification. Crucially, we have to be very careful regarding the usage of negation over these atoms, as we aim to utilize the GL reduct to model universal quantification. Hence, if we are not careful with the usage of negation, one could accidentally “remove” atoms from universal quantification. We briefly provide a simple saturation encoding to clarify the concept.

Example 3.4. Let $\exists a, b, \forall x, y. (a \wedge b \wedge \neg x) \vee (b \wedge \neg y) \vee y$ be a quantified Boolean formula (QBF) [6,100,101]. This is an extension of a propositional formula, where variables are quantified existentially or universally. It is known that deciding the consistency of such a QBF with two alternating quantifier blocks is Σ_2^P -complete. We will reduce from this formula to demonstrate how ASP can encode QBFs with two quantifier blocks. Then, we construct the following saturation encoding that computes all assignments α over $\{a, b\}$ such that for every assignment β over $\{x, y\}$ the formula $\varphi[\alpha][\beta]$ is satisfied.

We translate this formula to ASP by creating a logic program Π as follows. First, for every $v \in \{a, b, x, y\}$ we use atoms v and $\bar{v}$ and guess the truth value via rule $v \vee \bar{v} \leftarrow$. Intuitively, this means that a variable is either assigned to true or to false. We check satisfiability of a term by three rules: $\text{sat} \leftarrow a, b, \bar{x}$, $\text{sat} \leftarrow b, \bar{y}$, and $\text{sat} \leftarrow y$. Finally, we avoid unsatisfied formulas by $\leftarrow \neg \text{sat}$. and saturate universal variables by rules $x \leftarrow \text{sat}$, $\bar{x} \leftarrow \text{sat}$, $y \leftarrow \text{sat}$, and $\bar{y} \leftarrow \text{sat}$. Note that only $x, \bar{x}, y, \bar{y}$ are subject to saturation (assignments over universal variables).

One answer set of Π is $M = \{a, b, \text{sat}, x, \bar{x}, y, \bar{y}\}$. Indeed, observe that there is no smaller model of Π^M than M itself, which proves that the QBF is valid. However, $M' = \{a, \bar{b}, \text{sat}, x, \bar{x}, y, \bar{y}\}$ is not an answer set of Π , as $N \subsetneq M'$ with $N = \{a, \bar{b}, \bar{x}, \bar{y}\}$ is a smaller model of $\Pi^{M'}$. Observe how “saturating” the answer set via sat drives the search for smaller reduct models that do not contain sat .

3.2. Non-ground answer-set programming

Similar to ground ASP, we define non-ground ASP as follows. Let $p_1, \dots, p_n$ be predicates, where for $1 \leq i \leq n$ we denote by $|p_i|$ the *arity* of p_i , i.e., how many variables p_i takes as arguments. Note that $|p_i| \geq 0$. A *non-ground program* Π is a set of *rules* of the form

$$p_1(X_1) \vee \dots \vee p_\ell(X_\ell) \leftarrow p_{\ell+1}(X_{\ell+1}), \dots, p_m(X_m), \neg p_{m+1}(X_{m+1}), \dots, \neg p_n(X_n), \quad (4)$$

where for every *variable vector* X_i we have $|X_i| = |p_i|$. Whenever we have $x \in \langle X_1, \dots, X_\ell, X_{m+1}, \dots, X_n \rangle$, then $x \in \langle X_{\ell+1}, \dots, X_m \rangle$, such that rules can be considered *safe*, i.e., all variables that are used in a rule appear in some positive literal in the body. For a non-ground rule r , we let similarly to ground rules $H_r := \{p_1(X_1), \dots, p_\ell(X_\ell)\}$, $B_r^+ := \{p_{\ell+1}(X_{\ell+1}), \dots, p_m(X_m)\}$, $B_r^- := \{p_{m+1}(X_{m+1}), \dots, p_n(X_n)\}$, and additionally $\text{var}(r) := \{x \in X \mid p(X) \in H_r \cup B_r^+ \cup B_r^-\}$. Further, we use $\text{heads}(\Pi) := \{p(X) \in H_r \mid r \in \Pi\}$ to refer to the predicates (including the variable vector), and $\text{hpreds}(\Pi) := \{p \mid p(X) \in \text{heads}(\Pi)\}$ for the predicate names only, occurring in the head of a rule. Without loss of generality, we assume that *variables are unique per rule*, i.e., for every two rules $r, r' \in \Pi$, we have $\text{var}(r) \cap \text{var}(r') = \emptyset$. The rule size corresponds to $\|r\| := |B_r^+| + |B_r^-| + |H_r|$ and program size $\|\Pi\| := \sum_{r \in \Pi} \|r\|$. The attributes *disjunctive* and *normal*, as earlier defined for ground programs in Section 3.1, naturally carry over to non-ground rules and programs. Non-ground, *tight* programs can be defined as follows.

Definition 3.4. (Non-ground dependency graph) The *dependency graph* D_Π of a non-ground program Π is the directed graph defined on the set $\text{hpreds}(\Pi)$ of head predicates of the rules $r \in \Pi$. There is a directed edge from p to q whenever there is a rule $r \in \Pi$ with $p(X) \in B_r^+$ and $q(Y) \in H_r$.

A non-ground program Π is called *tight* if D_Π has no directed cycle.

In order to solve a given non-ground program Π , the *grounding* of the program, the process of instantiating the non-ground rules, is required. However, for the ease of presentation, a non-ground program does not contain constant symbols. Therefore, for the process of grounding Π , we require a set $\mathcal{F}(\Pi)$ of atoms, called *facts*, which are (true) atoms of the form $p(D)$ that are supposed to be contained in every answer set, with p being a predicate of Π and D being a vector of (function-free) constants of size $|D| = |p|$. We say that D is part of the *domain* of Π , defined by $\text{dom}(\Pi) := \{d \in D \mid p(D) \in \mathcal{F}(\Pi)\}$. We refer to the *set of all domain vectors* over $\text{dom}(\Pi)$ for a

³ Actually, researchers started calling this approach “the saturation technique” after the hardness proof was published [22]. However, this approach is still just a plain ASP encoding that correctly utilizes the complexity of Σ_2^P .

variable vector X of size $|X|$ by $\text{dom}(X)$. Let D be a domain vector over variable vector X and vector Y contain only variables of X . Then, the *domain vector of D restricted to Y* is given by D_Y .

Definition 3.5 (Grounding). The *grounding* $G(\Pi)$

consists of $F(\Pi)$

and ground rules obtained by replacing each rule r of Form (4) for every domain vector $D \in \text{dom}(\langle \text{var}(r) \rangle)$ by

$$p_1(D_{X_1}) \vee \dots \vee p_\ell(D_{X_\ell}) \leftarrow p_{\ell+1}(D_{X_{\ell+1}}), \dots, p_m(D_{X_m}), \\ \neg p_{m+1}(D_{X_{m+1}}), \dots, \neg p_n(D_{X_n}).$$

The effort of grounding a program Π is therefore in $\mathcal{O}(|\text{dom}(\Pi)|^n)$, where n is the maximum number of distinct variables over all rules.

Interestingly, for fixed arity the complexity of deciding consistency of a non-ground program increases only by one level.

Proposition 3.2 (Grounding Complexity for Fixed Arity [26]). *Let Π be any tight (normal) or disjunctive, non-ground program, where every predicate has arity at most a , for some natural number $a \geq 0$. Then, deciding whether $G(\Pi)$ admits an answer set is complete for Σ_2^P or Σ_3^P , respectively.*

We can note that, unless using growing predicate arities, with non-ground ASP we cannot encode problems above the polynomial hierarchy like PSPACE-complete problems (under standard complexity-theoretic assumptions). Further, these complexity results motivate avoiding the grounding of non-ground programs with bounded predicate arity by polynomially mapping or reducing them to ground programs [26].

Example 3.5. Consider the tight, non-ground program $\Pi := \{r\}$ with $r = a(x, y) \leftarrow b(x), c(y, z)$ and $F(\Pi) := \{b(1), c(1, 2), c(2, 1)\}$. Observe that, while $\text{dom}(\Pi) = \{1, 2\}$, for x we only need to consider a subset $\{1\} \subseteq \text{dom}(x)$ of the domain, as $x = 2$ can never happen, as there is no possibility to obtain $b(2)$. Therefore, and to keep our examples small, we assume for this program that $\text{dom}(x)$ were only $\{1\}$. In an implementation one could let the user specify the domain restriction, or better: perform efficient domain inference [102]. Then, if $\text{dom}(x)$ were $\{1\}$, our idealized grounding $P = G(\Pi)$ of Π consists of:

$$F(\Pi) \cup \{a(1, 1) \leftarrow b(1), c(1, 1), a(1, 1) \leftarrow b(1), c(1, 2), \\ a(1, 2) \leftarrow b(1), c(2, 1), a(1, 2) \leftarrow b(1), c(2, 2)\}.$$

The only answer set of P is $\{b(1), c(1, 2), c(2, 1), a(1, 1), a(1, 2)\}$.

3.3. Epistemic logic programming

An *epistemic logic program (ELP)* P is an extension of a logic program as defined above. The following definitions are based on [11, 13, 103, 104].

In addition to standard logic program, each rule body can contain *epistemic literals*. An epistemic literal is a formula of the form $\text{not } \ell$, where ℓ is a literal in the classical sense and **not** is the *epistemic negation* operator.

Let k, m, j, n be non-negative integers such that $k \leq m \leq j \leq n$ and $a_1, \dots, a_n$ be distinct propositional atoms. An *epistemic logic program (ELP)* is a set P of *ELP rules* of the form

$$a_1 \vee \dots \vee a_k \leftarrow \ell_{k+1}, \dots, \ell_m, \xi_{m+1}, \dots, \xi_j, \neg \xi_{j+1}, \dots, \neg \xi_n. \quad (5)$$

where each ℓ_i is a literal over atom a_i , and each ξ_i is an epistemic literal of the form $\text{not } \ell_i$, where ℓ_i is a literal over atom a_i . Similarly to logic programs, let $H_r = \{a_1, \dots, a_k\}$, and let $B_r = \{\ell_1, \dots, \ell_m, \xi_1, \dots, \xi_j, \neg \xi_{j+1}, \dots, \neg \xi_n\}$, i.e., the set of elements appearing in the rule body.

Then, $\text{at}(r) := \{a_1, \dots, a_n\}$ denotes the set of atoms occurring in an ELP rule r , which naturally extends to ELP programs. In a rule we sometimes write $\mathbf{K}\ell$ and $\mathbf{M}\ell$ for a literal ℓ , which refer to the expressions $\neg \text{not } \ell$ and $\text{not } \neg \ell$, respectively.

Definition 3.6 (World View Interpretation). Given an ELP P , a *world view interpretation (WVI)* I for P is a consistent set I of literals over a set $A \subseteq \text{at}(P)$ of atoms, i.e., $I \subseteq \{a, \neg a \mid a \in A\}$ such that there is no $a \in A$ with $\{a, \neg a\} \subseteq I$.

Intuitively, for a WVI I every $\ell \in I$ is considered as “known” and every $a \in A$ with $\{a, \neg a\} \cap I = \emptyset$ is treated as “possible”. We denote the WVI over a set $X \subseteq \text{at}(P)$ of atoms obtained by restricting I to X by $I|_X := I \cap \{a, \neg a \mid a \in X\}$. Next, we define compatibility with a set of interpretations.

Definition 3.7 (WVI Compatibility).

Let I be a set of interpretations over a set A of atoms. Then, a WVI I is *compatible* with I if:

1. $I \neq \emptyset$;
2. for each atom $a \in I$, it holds that for each $J \in I, a \in J$;
3. for each $\neg a \in I$, we have for each $J \in I, a \notin J$;
4. for each atom $a \in A$ with $\{a, \neg a\} \cap I = \emptyset$, there are $J, J' \in I$, such that $a \in J$, but $a \notin J'$.

While there are many semantics for ELPS [11–13,104], we will use the approach by Morak [105] in order to define the semantics of an ELP. His approach [105] follows the semantics defined in [13], but uses a different formal representation. Note that, however, our results can be adapted to other “reduct-based” semantics, by changing the definition of the reduct appropriately.

Definition 3.8. (Epistemic Reduct) The epistemic reduct [11] of program P w.r.t. a WVI I over a set A of atoms, denoted P^I , is defined as

$P^I = \{r^I \mid r \in \Pi\}$ where r^I denotes rule r where each epistemic literal $\text{not } \ell$, whose atom is also in A , is replaced by $\perp$ if $\ell \in I$, and by $\top$ otherwise. Note that P^I is a plain logic program with all occurrences of epistemic negation removed.

Then, a WVI I over $\text{at}(P)$ is a *world view* (WV) of P iff I is compatible with the set $AS(P^I)$. The set of WVs of an ELP P is denoted $WVS(P)$.

Of course, as an ASP extension, ELPS can also be encoded in a non-ground form, which requires grounding before the solving of a program. This paper will only cover ground ELPS, which is why we restrict the background to this category. The problem of deciding the existence of a world view for a ground ELP is known to be Σ_3^P -complete [103], as seen in Table 2.

Example 3.6. Consider the ground ELP program

$$P := \{a \vee b, g \leftarrow \neg \mathbf{K}a, a \leftarrow \neg \mathbf{K}g.\}$$

When constructing a WVI I over $\text{at}(P)$ one guesses for each atom $a \in \text{at}(P)$ either (1) $a \in I$, (2) $\neg a \in I$, or (3) $\{a, \neg a\} \cap I = \emptyset$ as described earlier, i.e., for the two atoms in $\text{at}(P)$ we obtain 3^3 possibilities. Each WVI I can be checked with the corresponding epistemic reduct P^I by verifying Definition 3.7 for $AS(P^I)$.

Consider $I_1 = \{a, \neg g, \neg b\}$ with its epistemic reduct $P^{I_1} := \{a \vee b, g \leftarrow \perp, a.\}$. Note that the epistemic reduct is indeed a plain logic program by the semantics of logic programs, as rules r with $\perp \in B_r^+$ or $\top \in B_r^-$ can obviously be dropped. Since $AS(P^{I_1}) = \{\{a\}\}$, compatibility of I_1 can be checked trivially which validates I_1 as WV of P (the second WV being $I_2 = \{g\}$, as $AS(P^{I_2}) = \{\{g, a\}, \{g, b\}\}$).

4. Body-decoupled grounding for tight ASP

This section introduces a novel grounding approach we call *Body-Decoupled Grounding*. With this approach we exploit the coinciding complexity of non-ground, tight programs, and disjunctive ground programs (cf. Table 1 ①) via a reduction $\mathcal{R}$. This reduction is encoded in such a way that body atoms are decoupled, i.e., the grounding effort is split up, effectively minimizing the combinatorial effort of instantiating variables compared to grounding the body as a whole. We first introduce the idea of Body-Decoupled Grounding for tight programs in Section 4.1, and later present improvements that allow us to decrease the number of resulting ground rules in Section 4.2.

It is in fact easy to see that the expressiveness increase from normal to disjunctive programs is inevitable (already for fixed arity).

Proposition 4.1. (*Disjunctive Programs Inevitable*) Let Π be any non-ground, tight program, where every predicate has arity at most a . Then, unless $\text{NP} = \Sigma_2^P$, there cannot be a polynomial grounding procedure R' such that $R'(\Pi)$ is normal.

Proof. While consistency for ground, normal programs is in NP, we know Σ_2^P -hardness for disjunctive (head-cycle-free) non-ground programs [26, Lemma 6]. The result therefore immediately extends to non-ground tight programs [91]. $\square$

4.1. Towards body-decoupled grounding

First, we present our approach for tight programs without disjunction. To this end, we assume a given non-ground, tight program Π and a set $\mathcal{F}(\Pi)$ of facts. Then, our reduction relies on the following sets of variables. For each predicate $p(X)$ in $\text{heads}(\Pi)$, we use every instantiation of $p(X)$ and an auxiliary predicate $\bar{p}(X)$ that stands for the negation of $p(X)$ over $\text{dom}(\Pi)$, resulting in atoms AtPred , defined as follows:

$$\{p(D), \bar{p}(D) \mid p(X) \in \text{heads}(\Pi), D \in \text{dom}(X)\}$$

These atoms AtPred are used as a guess for (potential) answer sets. Note that the atoms over $\bar{p}(X)$ are used for technical reasons only, and are not needed when employing, e.g., choice rules [38].

In addition to AtPred and in accordance with the semantics of ASP, we ensure (i) satisfiability and (ii) foundedness. For (i), the computation of models of rules, we require atoms $\text{AtSat} := \{\text{sat}, \text{sat}_r, \text{gr}_x(d) \mid r \in \Pi, x \in \text{var}(r), d \in \text{dom}(x)\}$, where sat_r indicates the satisfiability of a non-ground rule r and sat the satisfiability of the program, respectively. Atoms of the form $\text{gr}_x(d)$ are used to indicate that we assign variable x of non-ground rule r to domain value $d \in \text{dom}(x)$ when checking for satisfiability.

For (ii), which ensures foundedness, we use variables AtUf , defined as follows:

$$\{\text{uf}_r(D) \mid r \in \Pi, D \in \text{dom}(X), h(X) \in H_r\} \cup \{\text{gr}_{y, D_{(X,y)}} \mid r \in \Pi, D \in \text{dom}(\langle \text{var}(r) \rangle), h(X) \in H_r, y \in \text{var}(r), y \notin X\}$$

Intuitively, $\text{uf}_r(D)$ indicates that r fails to found $h(D)$ for head predicate $h(X) \in H_r$ and domain vector $D \in \text{dom}(X)$, where $\text{gr}_{y, D_{(X,y)}}$ refers to the assigned domain value d that variable y gets in this foundedness check for $h(D)$, i.e., we ensure foundedness with atoms deriving unfoundedness. Overall, the number of atoms used for our approach is limited by the largest predicate arity as follows.

Guess Answer Set Candidates

for every $h(X) \in \text{heads}(\Pi)$, $D \in \text{dom}(X)$:

$$h(D) \vee \bar{h}(D) \leftarrow \quad (\mathcal{R}1)$$

Ensure Satisfiability

for every $r \in \Pi$, $x \in \text{var}(r)$:

$$\bigvee_{d \in \text{dom}(x)} \text{gr}_x(d) \leftarrow \quad (\mathcal{R}2)$$

for every $r \in \Pi$, $p(X) \in B_r^+$, $D \in \text{dom}(X)$, $X = \langle x_1, \dots, x_\ell \rangle$:

$$\text{sat}_r \leftarrow \text{gr}_{x_1}(D_{\langle x_1 \rangle}), \dots, \text{gr}_{x_\ell}(D_{\langle x_\ell \rangle}), \neg p(D) \quad (\mathcal{R}3)$$

for every $r \in \Pi$, $p(X) \in H_r \cup B_r^-$, $D \in \text{dom}(X)$, $X = \langle x_1, \dots, x_\ell \rangle$:

$$\text{sat}_r \leftarrow \text{gr}_{x_1}(D_{\langle x_1 \rangle}), \dots, \text{gr}_{x_\ell}(D_{\langle x_\ell \rangle}), p(D) \quad (\mathcal{R}4)$$

where $\Pi = \{r_1, \dots, r_n\}$:

$$\text{sat} \leftarrow \text{sat}_{r_1}, \dots, \text{sat}_{r_n} \quad (\mathcal{R}5)$$

for every $r \in \Pi$, $x \in \text{var}(r)$, $d \in \text{dom}(x)$:

$$\text{gr}_x(d) \leftarrow \text{sat} \quad (\mathcal{R}6)$$

$$\leftarrow \neg \text{sat} \quad (\mathcal{R}7)$$

Prevent Unfoundedness

for every $r \in \Pi$, $h(X) \in H_r$, $D \in \text{dom}(X)$, $y \in \text{var}(r)$, $y \notin X$:

$$\bigvee_{d \in \text{dom}(y)} \text{gru}_y(\langle D, d \rangle) \leftarrow h(D) \quad (\mathcal{R}8)$$

for every $r \in \Pi$, $h(X) \in H_r$, $p(Y) \in B_r^+$, $D \in \text{dom}(\langle X, Y \rangle)$, $Y \setminus X = \langle y_1, \dots, y_\ell \rangle$:

$$\text{uf}_r(D_X) \leftarrow \text{gru}_{y_1}(D_{\langle X, y_1 \rangle}), \dots, \text{gru}_{y_\ell}(D_{\langle X, y_\ell \rangle}), \neg p(D_Y) \quad (\mathcal{R}9)$$

for every $r \in \Pi$, $h(X) \in H_r$, $p(Y) \in B_r^- \cup (H_r \setminus \{h(X)\})$, $D \in \text{dom}(\langle X, Y \rangle)$, $Y \setminus X = \langle y_1, \dots, y_\ell \rangle$:

$$\text{uf}_r(D_X) \leftarrow \text{gru}_{y_1}(D_{\langle X, y_1 \rangle}), \dots, \text{gru}_{y_\ell}(D_{\langle X, y_\ell \rangle}), p(D_Y) \quad (\mathcal{R}10)$$

for every $h(X) \in \text{heads}(\Pi)$, $D \in \text{dom}(X)$, $\{r_1, \dots, r_m\} = \{r \in \Pi \mid h(X) \in H_r\}$:

$$\leftarrow h(D), \text{uf}_{r_1}(D), \dots, \text{uf}_{r_m}(D) \quad (\mathcal{R}11)$$

Fig. 2. Body-decoupled grounding procedure $\mathcal{R}$ for a given non-ground, tight program Π , which creates a disjunctive, *ground* program.

Observation 1. (Bounded Number of Auxiliary Atoms) The number of atoms in $|\text{AtPred} \cup \text{AtSat} \cup \text{AtUf}|$ is bounded by $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^{a+1})$, where a is the largest predicate arity.

Intuition of our reduction. After the declaration of the used atoms, we are in position to explain the intuition of our reduction for tight programs. The overall idea consists of three parts that together form the reduction $\mathcal{R}$, which transforms the non-ground, tight program Π into a ground, disjunctive program $\mathcal{R}(\Pi)$ consisting of facts $\mathcal{F}(\Pi)$ and the rules given in Fig. 2. Our reduction follows the basic characterization of answer sets of Definition 3.3, but we first restrict to tight programs for simplicity (for tight programs no orderings are required). Based on Definition 3.3, we therefore first need to find models of the program: Rules (R1) take care of guessing answer set candidates, then Rules (R2)–(R7) ensure (i) *satisfiability of all rules*, i.e., for every potential variable instantiation. Since this is a universal statement (intuitively, for every assignments of variables to domain elements the resulting ground rule must hold), this is where the saturation technique (see Section 3.1.1) will be applied. Then, we require foundedness of every atom part of the answer set candidate, which we encode indirectly via Rules (R8)–(R11) that aim at (ii) *preventing unfounded atoms*. The intuitive reason why we encode foundedness via (ii) indirectly, is that this way our approach supports body-decoupled grounding. Indeed, justifying foundedness requires to find a rule whose whole body is true (see also Definition 3.3 for details), but encoding that a single rule does not found a specific head predicate just requires a single body element to differ from the answer set candidate. While this is only an intuitive explanation, we provide a more formal proof of this concept below. Note that the foundedness check is not subject to saturation.

Interestingly, the only disjunction that is indeed crucial and cannot be modeled via choice rules is the disjunction part of Rules (R2) for (i) satisfiability, which is responsible for guessing and saturating assignments of variables to domain values. The construction is such that whenever for a non-ground rule $r \in \Pi$ there is an assignment of variables to domain values such that the resulting ground rule is satisfied, Rules (R3) or (R4) yield sat_r . If such an atom sat_r can be derived for all non-ground rules of $r \in \Pi$, we derive sat by Rules (R5), which is mandatory (cf. Rules (R7)). Then, Rules (R6) apply *saturation*, which causes the assignment of all domain values to every variable. Assuming that a grounding of a rule $r \in \Pi$ was not satisfied, then there would exist a $\subseteq$ -smaller model of the reduct invalidating the answer set candidate. Intuitively, the construction takes care that there is an answer set of $\mathcal{R}(\Pi)$, only if the grounding of Π admits an answer set.

Rules (R8) are for (ii) preventing unfoundedness, ensuring that for each head atom $h(D)$ contained in a model of $\mathcal{R}(\Pi)$, variables get assigned domain values for proving foundedness. Unfoundedness of a rule $r \in \Pi$ for atom $h(D)$ is derived by Rules (R9) and (R10), which is prevented by Rules (R11).

Example 4.1. Consider the non-ground, tight program $\Pi = \{a(x, y) \leftarrow b(x), c(y, z)\}$ and facts $F(\Pi) = \{b(1), c(1, 2), c(2, 1)\}$ from Example 3.5. Grounding the program as described above and shown in Fig. 2, results in the ground program $P = \mathcal{R}(\Pi)$, which will be shown in Fig. 3 below. The answer sets of P restricted to predicate symbols a, b and c of Π the unique answer set $\{a(1, 1), a(1, 2), b(1), c(1, 2), c(2, 1)\}$.

We prove the correctness of procedure $\mathcal{R}$ with the following theorem.

Theorem 4.1. (Correctness) *Let Π be any non-ground, tight normal program. Then, the grounding procedure $\mathcal{R}$ on Π is correct, i.e., the answer sets of $\mathcal{R}(\Pi)$ restricted to $\text{at}(\mathcal{G}(\Pi))$ match the answer sets of $\mathcal{G}(\Pi)$.*

Precisely, (1) for every answer set M of $\mathcal{G}(\Pi)$, there is an answer set M' of $\mathcal{R}(\Pi)$ with $M = M' \cap \text{at}(\mathcal{G}(\Pi))$.

Further, (2) for every answer set M' of $\mathcal{R}(\Pi)$, $M' \cap \text{at}(\mathcal{G}(\Pi))$ is an answer set of $\mathcal{G}(\Pi)$.

Proof. We prove Statements (1) and (2) independently. For the ease of presentation and without loss of generality, we assume that for every rule $r \in \Pi$ and $h(X) \in H_r$, there is at least one variable $y \in \text{var}(r)$ that is not in X , which can be easily achieved by adding a fresh fixed dummy variable to some body predicate.

(1): Let M be an answer set of $\mathcal{G}(\Pi)$ and assume towards a contradiction that there is no extension $M' \supseteq M$ of M with $M \cap \text{at}(\mathcal{G}(\Pi)) = M' \cap \text{at}(\mathcal{G}(\Pi))$ such that M' is an answer set of $\mathcal{R}(\Pi)$.

First, we construct $N := \{h(D) \mid h(X) \in \text{heads}(\Pi), D \in \text{dom}(X), h(D) \notin M\}$, which collects the complements of those head atoms. Further, we gather satisfied rules by $S := \{\text{sat}, \text{sat}_r, \text{gr}_x(d) \mid r \in \Pi, x \in \text{var}(r), d \in \text{dom}(x)\}$.

Then, we set appropriate domain values for the founded atoms:

$$\begin{aligned} F &:= \{\text{gru}_{y_1}(\langle D, D'_{y_1} \rangle), \dots, \text{gru}_{y_\ell}(\langle D, D'_{y_\ell} \rangle) \mid r \in \Pi, h(X) \in H_r, \\ &\quad D' \in \text{dom}(\langle \text{var}(r) \rangle), D = D'_X, \\ &\quad M \supseteq \{h(D)\} \cup \{p(D'_Z) \mid p(Z) \in B_r^+\}, \\ &\quad M \cap \{p(D'_Z) \mid p(Z) \in (B_r^- \cup H_r), p(Z) \neq h(X)\} = \emptyset, \\ &\quad \{y_1, \dots, y_\ell\} = \{y \in \text{var}(r) \mid y \notin X\} \}. \end{aligned}$$

Intuitively, atoms F are those that we need to establish foundedness of elements $h(D)$ in M for matching $h(X) \in H_r$. Note that therefore we must correctly instantiate our variables outside of X , which is what the definition of F ensures.

Finally, we set the domain values for unfounded atoms $U := \{\text{uf}_r(D), \text{gru}_y(\langle D, d(y) \rangle) \mid r \in \Pi, h(X) \in H_r, h(D) \in M, y \in \text{var}(r), y \notin X, \text{ there is no } \text{gru}_y(\langle D, d' \rangle) \in F\}$, where $d(y)$ yields any arbitrary, fixed domain value in $\text{dom}(y)$.

Then, we let $M' := M \cup N \cup F \cup U$. Since by assumption M' is not an answer set, either (i) one of the rules in $\mathcal{R}(\Pi)$ is not satisfied, or (ii) M' is not minimal.

We proceed by case distinction. Case (i): Some rule $r \in \mathcal{R}(\Pi)$ is not satisfied by M' . It is easy to see that all heads of Rules (R1)–(R7) are satisfied by construction of M' . Rules (R8) are satisfied by construction of F and U . The construction of U also ensures that Rules (R9) and (R10) are satisfied.

Observe that Rules (R11) are satisfied, since M is an answer set of $\mathcal{G}(\Pi)$ and therefore every atom of M is founded, as constructed by F . This concludes Case (i), since every rule of $\mathcal{R}(\Pi)$ is satisfied.

Case (ii): Model M' of $\mathcal{R}(\Pi)$ is not minimal, i.e., there exists a model $M'' \subsetneq M'$ with M'' being a model of $\mathcal{R}(\Pi)^{M'}$.

Note that the difference between M' and M'' cannot be due to Rule (R1), since then M' and M'' would be incomparable.

If $\text{sat} \notin M''$, then by Rules (R5), there is a rule $r_i \in \Pi$ with $\text{sat}_{r_i} \notin M''$, which by Rules (R3) and (R4) implies that $\mathcal{G}(\{r_i\})$ is not satisfied by M'' . This, however, contradicts the assumption that M is an answer set of $\mathcal{G}(\Pi)$.

Consequently, by Rules (R6), the difference between M' and M'' cannot be due to any gr_x predicate.

Further, M' and M'' cannot differ in any predicate of the form gru_y , either, since by Rules (R8), for every $h(D) \in M'$, there is precisely one $\text{gru}_y(\langle D, d \rangle) \in M'$.

Similarly, Rules (R9)–(R11) yield deterministic consequences, depending only on the choice of Rules (R1) and (R8). As a result, we conclude that $M' = M''$.

(2): Let M' be an answer set of $\mathcal{R}(\Pi)$ and assume towards a contradiction that $M := M' \cap \text{at}(\mathcal{G}(\Pi))$ is not an answer set of $\mathcal{G}(\Pi)$.

Then, either (i) M is not a model of $\mathcal{G}(\Pi)$, or (ii) there is an $h(D) \in M$ that is unfounded by M . We proceed by case distinction.

Case (i): M is not a model of $\mathcal{G}(\Pi)$. Assume towards a contradiction that there is a rule $r_i \in \Pi$ with M not satisfying $\mathcal{G}(\{r_i\})$. Then, there is a subset $M'' \subsetneq M'$ with $\text{sat}_{r_i} \notin M''$, $\text{sat} \notin M''$ and gr_x predicate set accordingly for $x \in \text{var}(r_i)$, that is a model of $\mathcal{R}(\Pi)$, which contradicts that M' is an answer set ($\subseteq$ -minimal model of $\mathcal{R}(\Pi)^{M'}$).

Case (ii): There exists an $h(D) \in M$ that is unfounded by M . Then, no matter how the predicates gru_x are assigned in M' , since $h(D) \in M$ is not founded, there is no rule $r_i \in \Pi$ that can be instantiated such that the body is satisfied and thereby founding $h(D)$. Consequently, for every such rule r_i , by Rules (R9) and (R10), there is at least one atom over predicates uf_{r_i} in M' .

Finally, Rules (R11) are not satisfied (for $h(D)$), which contradicts that M' is an answer set of $\mathcal{R}(\Pi)$. $\square$

Note that the definition of U in the proof above indeed suggests that there is freedom in the assignment of variables. While it is not immediately clear how this could be improved, it seems that there is potential for future (practical) improvements of Rules (R8), which requires that variables are instantiated for the unfounded check (but could get a default value in some cases).

$$\begin{aligned}
& a(1, 1) \vee \bar{a}(1, 1). \quad a(1, 2) \vee \bar{a}(1, 2). && \text{By } (\mathcal{R}1) \\
& \text{gr}_x(1) \cdot \text{gr}_y(1) \vee \text{gr}_y(2) \cdot \text{gr}_z(1) \vee \text{gr}_z(2). && \text{By } (\mathcal{R}2) \\
& \text{sat}_r \leftarrow \text{gr}_x(1), \neg b(1). && \text{By } (\mathcal{R}3) \\
& \text{sat}_r \leftarrow \text{gr}_y(1), \text{gr}_z(1), \neg c(1, 1). \quad \text{sat}_r \leftarrow \text{gr}_y(1), \text{gr}_z(1), \neg c(1, 2). \\
& \text{sat}_r \leftarrow \text{gr}_y(2), \text{gr}_z(2), \neg c(2, 1). \quad \text{sat}_r \leftarrow \text{gr}_y(2), \text{gr}_z(2), \neg c(2, 2). \\
& \text{sat}_r \leftarrow \text{gr}_x(1), \text{gr}_y(1), a(1, 1). && \text{By } (\mathcal{R}4) \\
& \text{sat}_r \leftarrow \text{gr}_x(1), \text{gr}_y(2), a(1, 2). \\
& \text{sat} \leftarrow \text{sat}_r. && \text{By } (\mathcal{R}5) \\
& \text{gr}_x(1) \leftarrow \text{sat}. \quad \text{gr}_y(1) \leftarrow \text{sat}. \quad \text{gr}_y(2) \leftarrow \text{sat}. && \text{By } (\mathcal{R}6) \\
& \text{gr}_z(1) \leftarrow \text{sat}. \quad \text{gr}_z(2) \leftarrow \text{sat}. \\
& \leftarrow \neg \text{sat}. && \text{By } (\mathcal{R}7) \\
& \text{gru}_z(1, 1, 1) \vee \text{gru}_z(1, 1, 2) \leftarrow a(1, 1). \quad \text{gru}_z(1, 2, 1) \vee \text{gru}_z(1, 2, 2) \leftarrow a(1, 2). \\
& && \text{By } (\mathcal{R}8) \\
& \text{uf}_r(1, 1) \leftarrow \neg b(1). \quad \text{uf}_r(1, 2) \leftarrow \neg b(1). && \text{By } (\mathcal{R}9) \\
& \text{uf}_r(1, 1) \leftarrow \neg c(1, 1), \text{gru}_z(1, 1, 1). \quad \text{uf}_r(1, 1) \leftarrow \neg c(1, 2), \text{gru}_z(1, 1, 2). \\
& \text{uf}_r(1, 2) \leftarrow \neg c(2, 1), \text{gru}_z(1, 2, 1). \quad \text{uf}_r(1, 2) \leftarrow \neg c(2, 2), \text{gru}_z(1, 2, 2). \\
& \leftarrow a(1, 1), \text{uf}_r(1, 1). \quad \leftarrow a(1, 2), \text{uf}_r(1, 2). && \text{By } (\mathcal{R}11)
\end{aligned}$$

Fig. 3. $\mathcal{R}(\Pi)$ with Π from Example 4.1, guided by $\mathcal{R}$ of Fig. 2.

The reduction $\mathcal{R}$ works in polynomial time, since our technique does not suffer from large rules (or large rule bodies).

Theorem 4.2 (Polynomial runtime and grounding size). *Let Π be any non-ground, tight program, where every predicate has arity at most a . Then, the grounding procedure $\mathcal{R}$ on Π is polynomial, i.e., can be computed in time $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^{2-a})$.*

Proof. The reduction $\mathcal{R}$ constructs $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^a)$ many Rules ($\mathcal{R}1$) of constant size, $\mathcal{O}(\|\Pi\| \cdot a)$ many Rules ($\mathcal{R}2$) of size $|\text{dom}(\Pi)|$, as well as $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^a)$ many Rules ($\mathcal{R}3$) and ($\mathcal{R}4$) of size $\mathcal{O}(a)$. Then, there is one Rule ($\mathcal{R}5$) of size $\mathcal{O}(|\Pi|)$, $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|)$ many Rules ($\mathcal{R}6$) and one Rule ($\mathcal{R}7$). Finally, there are $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^a \cdot a)$ many Rules ($\mathcal{R}8$) of size $\mathcal{O}(|\text{dom}(\Pi)| \cdot a)$, and we require $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^{2-a})$ effort for Rules ($\mathcal{R}9$) and ($\mathcal{R}10$), as well as $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|)$ effort for Rules ($\mathcal{R}11$). $\square$

4.2. Improvements

The reduction $\mathcal{R}$, as described above, can also be further improved. In the following we introduce such improvements that decrease the number of resulting ground rules and improve practicality

Partial reducibility. While the idea of decoupling rule bodies might seem tempting in theory, it is unlikely that the approach will outperform grounders that were highly optimized over years, especially for simpler problem encodings. Conscious of what optimized grounders are capable of, our approach can be optimized such that only tough rules are reduced by our reduction (i.e., those with large bodies). Notably, with our approach the program can be split and the reduction $\mathcal{R}$ just partially applied, as long as foundedness can still be provided. We back this statement with the following corollary.

Corollary 4.1 (Partial reducibility). *Let Π be a non-ground, tight program Π and Π_1, Π_2 be a partition of Π such that $\text{hpreds}(\Pi_1) \cap \text{hpreds}(\Pi_2) = \emptyset$. Then, the answer sets of $\mathcal{R}(\Pi_1) \cup \mathcal{G}(\Pi_2)$ restricted to $\text{at}(\mathcal{G}(\Pi))$, where $\mathcal{R}(\Pi_1)$ is constructed over $\text{dom}(\Pi)$ and $\mathcal{G}(\Pi_2)$ is grounded over $\text{dom}(\Pi)$, match those of $\mathcal{G}(\Pi)$.*

Proof. (Sketch) The result is a direct consequence of the proof construction of Theorem 4.1, which can be extended.

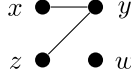
Satisfiability checking of $\mathcal{R}(\Pi_1) \cup \mathcal{G}(\Pi_2)$ works similarly, since satisfiability of rules in $\mathcal{G}(\Pi_1)$ are ensured by Rules ($\mathcal{R}2$)–($\mathcal{R}7$), and satisfiability of rules $\mathcal{G}(\Pi_2)$ is treated directly.

Since $\text{hpreds}(\Pi_1) \cap \text{hpreds}(\Pi_2) = \emptyset$, predicates in $\text{hpreds}(\Pi_1)$, $\text{hpreds}(\Pi_2)$ can only appear in rule bodies of Π_2 , Π_1 , respectively.

Consequently, for a given model of $\mathcal{G}(\Pi)$, the foundedness of atoms over predicates $\text{hpreds}(\Pi_1)$ is decided by $\mathcal{R}(\Pi_1)$, namely Rules ($\mathcal{R}8$)–($\mathcal{R}11$), whereas foundedness of atoms over $\text{hpreds}(\Pi_2)$ is checked by $\mathcal{G}(\Pi_2)$. $\square$

Thereby, partial reducibility not only allows for a more broad field of application, but also for interaction of sophisticated grounders and the reduction $\mathcal{R}$. As this implies, while the reduction can handle harder rules with large rule bodies, traditional grounders can still be used for simpler rule sets.

Example 4.2. Consider the non-ground, tight program $\Pi_2 := \Pi \cup \{r_2\}$ with $r_2 = d(w) \leftarrow b(w)$, and program Π and facts $\mathcal{F}(\Pi) = \{b(1), c(1, 2), c(2, 1)\}$ from Example 4.1. One can split the program Π_2 into program parts $\Pi_{2,1} := \{a(x, y) \leftarrow b(x), c(y, z)\}$ and $\Pi_{2,2} :=$

Fig. 4. Variable Graph $\mathcal{V}_{\Pi_2}$ of Π_2 .**Improved Foundedness, replacing (R9)–(R11) of $\mathcal{R}$**

$$\text{for every } r \in \Pi, h(X) \in H_r, p(Y) \in B_r^+, D \in \text{dom}(\langle X, Y \rangle), Y \setminus X = \langle y_1, \dots, y_\ell \rangle: \quad (\mathcal{R}'9)$$

$$\text{uf}_{\text{rch}(Y, X)}(D_{\langle \text{rch}(Y, X) \rangle}) \leftarrow \text{gru}_{y_1}(D_{\langle X, y_1 \rangle}), \dots, \text{gru}_{y_\ell}(D_{\langle X, y_\ell \rangle}), \neg p(D_Y)$$

$$\text{for every } r \in \Pi, h(X) \in H_r, p(Y) \in B_r^- \cup (H_r \setminus \{h(X)\}), D \in \text{dom}(\langle X, Y \rangle), Y \setminus X = \langle y_1, \dots, y_\ell \rangle: \quad (\mathcal{R}'10)$$

$$\text{uf}_{\text{rch}(Y, X)}(D_{\langle \text{rch}(Y, X) \rangle}) \leftarrow \text{gru}_{y_1}(D_{\langle X, y_1 \rangle}), \dots, \text{gru}_{y_\ell}(D_{\langle X, y_\ell \rangle}), p(D_Y)$$

$$\text{for every } h(X) \in \text{heads}(\Pi), D \in \text{dom}(X), \{r_1, \dots, r_m\} = \{r \in \Pi \mid h(X) \in H_r\}: \quad (\mathcal{R}'11)$$

$$\leftarrow h(D), \left[\bigvee_{p(Y) \in B_{r_1}} \text{uf}_{\text{rch}(Y, X)}(D_{\langle \text{rch}(Y, X) \rangle}) \right], \dots, \left[\bigvee_{p(Y) \in B_{r_m}} \text{uf}_{\text{rch}(Y, X)}(D_{\langle \text{rch}(Y, X) \rangle}) \right]$$

Fig. 5. More involved formalization of checking for unfounded atoms based on [Observation 2](#), yielding alternative $\mathcal{R}'$ (cf. $\mathcal{R}$ of [Fig. 2](#)).

$\{d(w) \leftarrow b(w)\}$ to achieve partial reducibility, since $\text{hpreds}(\Pi_{2,1}) \cap \text{hpreds}(\Pi_{2,2}) = \emptyset$. Then, when grounding subprogram $\Pi_{2,1}$ using the reduction $\mathcal{R}$ and grounding $\Pi_{2,2}$ via a traditional grounder $\mathcal{G}$, the results $\mathcal{R}(\Pi_{2,1})$ and $\mathcal{G}(\Pi_{2,2})$ can be joined resulting in the rules shown in [Fig. 3](#) with an additional rule for $\mathcal{G}(\Pi_{2,2}) = \{d(1) \leftarrow b(1)\}$. Now, when involving facts $\mathcal{F}(\Pi)$, one will end up with the answer set $\{a(1, 1), a(1, 2), b(1), c(1, 2), c(2, 1), d(1)\}$, as expected.

Utilizing variable independencies. When analyzing the established grounding procedure $\mathcal{R}$ and examples (cf. [Example 4.1](#)), we can identify redundancies in the rule set for checking for unfoundedness of non-ground rules. Especially, in cases where variables of the head predicate of the non-ground rule do not occur in some body predicate, multiple rules are produced to cover the whole possible ground atom set.

Based on that consideration, we provide an improvement of the grounding procedure $\mathcal{R}$ utilizing reachable paths of the *variable graph*, which might significantly reduce the number of atoms for the foundedness check. To do so, we define the variable graph $\mathcal{V}_\Pi$ of a non-ground program Π in the following.

Definition 4.1. (Variable graph) The variable graph $\mathcal{V}_\Pi$ of Π is the graph defined on the set $\text{var}(r)$ of variables of rules $r \in \Pi$, where two variables $x, y \in \text{var}(r)$ are joined by an edge (x, y) whenever there is a predicate $p(X)$ in r with $x, y \in X$. Let X, Y be variable vectors. We refer by $\text{rch}(Y, X)$ to those vertices in X that are *reachable* by some $y \in Y$ in $\mathcal{V}_\Pi$.

To this end, we rely on the following observation.

Observation 2. (Variable-justification independency) Let Π be a non-ground program and let $r \in \Pi$ be a rule with $h(X) \in H_r$ and $p(Y) \in B_r^+$. Further, let I be a set of atoms over $\mathcal{G}(\Pi)$. Then, if r does not found $h(D_X) \in I$ for $D \in \text{dom}(\langle X, Y \rangle)$ due to $p(D_Y) \notin I$, r fails to found any $h(D') \in I$ with $D' \in \text{dom}(X)$ and $D'_{\langle \text{rch}(Y, X) \rangle} = D_{\langle \text{rch}(Y, X) \rangle}$ as well.

Example 4.3. Consider the non-ground, tight program Π_2 along with its facts $\mathcal{F}(\Pi) = \{b(1), c(1, 2), c(2, 1)\}$ taken from [Example 4.2](#). The variable graph $\mathcal{V}_{\Pi_2}$, shown in [Fig. 4](#), connects variables appearing together in atoms. In r_1 , variables x and y co-occur in $a(x, y)$, and y and z co-occur in $c(y, z)$, forming a chain $x-y-z$ in $\mathcal{V}_{\Pi_2}$. Variable w from r_2 remains disconnected.

Now, consider rule $r_1 = a(x, y) \leftarrow b(x), c(y, z)$ and an interpretation I such that $b(1) \in I$, but $c(2, 1) \notin I$. Then r_1 does not found $a(1, 2) \in I$. By [Observation 2](#), this failure propagates to any $a(D') \in I$ with $D'_x = 1$ and $D'_y = 2$.

Based on the observation above, we provide an alternative $\mathcal{R}'$ of reduction $\mathcal{R}$ (see [Fig. 5](#)), which is optimized to consider independent variables according to [Observation 2](#). Instead of AtUf , we use atoms $\text{AtUfI} :=$

$$\{\text{uf}_{\text{rch}(Y, X)}(D) \mid r \in \Pi, D \in \text{dom}(\langle \text{rch}(Y, X) \rangle), h(X) \in H_r, p(Y) \in B_r^+ \cup B_r^- \} \cup$$

$$\{\text{gru}_y(D_{\langle X, y \rangle}) \mid r \in \Pi, D \in \text{dom}(\langle \text{var}(r) \rangle), h(X) \in H_r, p(Y) \in B_r^+ \cup B_r^-, y \in Y \setminus X\}.$$

The updated reduction $\mathcal{R}'$ is given in [Fig. 5](#), where Rules (R'9) and (R'10) replace Rules (R9) and (R10). The only difference is the head predicate with a potentially smaller domain vector. Further, Rules (R11) are replaced by Rules (R'11), which contain disjunctions in their bodies, as in the well-known weight rules [38,106]. Alternatively, one can build plain rules by creating the cross product among the sets of disjuncts of Rules (R'11).

Example 4.4. Consider the non-ground, tight program Π of [Example 4.1](#). Based on the [Observation 2](#), the rules for preventing unfoundedness can be improved and reduced by incorporating variable independencies, resulting in fewer rules for ensuring foundedness, as shown in [Fig. 6](#).

Comparing to [Fig. 3](#), one can identify that Rules (R'9)–(R'11) do in fact reduce the number of necessary instantiations and rules. While the head predicate of the only rule r in the program Π contains the two variables x and y , they are only partially incorporated in the (decoupled) body-predicates b and c of r . Therefore, the rule set (R'9) can be reduced to the domain vector only containing

$$\begin{aligned}
& \text{uf}_{\{x\}}(1) \leftarrow \neg b(1). && \text{By } (\mathcal{R}'9) \\
& \text{uf}_{\{y\}}(1) \leftarrow \neg c(1, 1), \text{gru}_z(1, 1). \text{uf}_{\{y\}}(1) \leftarrow \neg c(1, 2), \text{gru}_z(1, 2). \\
& \text{uf}_{\{y\}}(2) \leftarrow \neg c(2, 1), \text{gru}_z(2, 1). \text{uf}_{\{y\}}(2) \leftarrow \neg c(2, 2), \text{gru}_z(2, 2). \\
& \leftarrow a(1, 1), [\text{uf}_{\{x\}}(1) \vee \text{uf}_{\{y\}}(1)]. && \text{By } (\mathcal{R}'11) \\
& \leftarrow a(1, 2), [\text{uf}_{\{x\}}(1) \vee \text{uf}_{\{y\}}(2)].
\end{aligned}$$

Fig. 6. Optimized foundedness check using the reduction $\mathcal{R}'$ for Π from Example 4.1.

Additional Rules for Foundedness of Normal Programs

$$\begin{aligned}
& \text{for every } p(X), p'(X') \in \text{heads}(\Pi), D \in \text{dom}(X), D' \in \text{dom}(X'), p(D) \neq p'(D'): \\
& [p(D) \prec p'(D')] \vee [p'(D') \prec p(D)] \leftarrow && (\mathcal{R}''12)
\end{aligned}$$

$$\begin{aligned}
& \text{for every } p_1(X_1), p_2(X_2), p_3(X_3) \in \text{heads}(\Pi), D_1 \in \text{dom}(X_1), D_2 \in \text{dom}(X_2), D_3 \in \text{dom}(X_3), \\
& \quad p_1(D_1) \neq p_2(D_2), p_1(D_1) \neq p_3(D_3), p_2(D_2) \neq p_3(D_3): \\
& \leftarrow [p_1(D_1) \prec p_2(D_2)], [p_2(D_2) \prec p_3(D_3)], [p_3(D_3) \prec p_1(D_1)] && (\mathcal{R}''13)
\end{aligned}$$

$$\begin{aligned}
& \text{for every } r \in \Pi, h(X) \in H_r, p(Y) \in B_r^+, D \in \text{dom}(\langle X, Y \rangle), Y \setminus X = \langle y_1, \dots, y_\ell \rangle, p(D_Y) \notin \mathcal{F}(\Pi): \\
& \text{uf}_r(D_X) \leftarrow \text{gru}_{y_1}(D_{\langle X, y_1 \rangle}), \dots, \text{gru}_{y_\ell}(D_{\langle X, y_\ell \rangle}), \neg[p(D_Y) \prec h(D_X)] && (\mathcal{R}''14)
\end{aligned}$$

Fig. 7. For a given non-ground, normal program Π , we require *additional rules* for ensuring foundedness, yielding $\mathcal{R}''$.

variable x when checking for predicate b and, similar, to only y when checking for c . In this way, the number of instantiations can be reduced to the combinations of variables actually affecting the body-predicate. To incorporate these changes and the different occurrences of variables in the head predicate uf_{rch} of Rules $(\mathcal{R}'9)$ and $(\mathcal{R}'10)$, we switch in Rules $(\mathcal{R}'11)$ to weighted disjunctions of these predicates in the body.

5. Applying body-decoupled grounding for other program types

In this section, we will lift the idea of body-decoupled grounding to other program types. This can be seen as an extension to the previous approach where, due to the program's complexity, additional effort is required.

5.1. Body-decoupled grounding for normal ASP

First, we lift the idea of our reduction $\mathcal{R}$ to non-ground, normal programs. To this end, one can rely on orderings (level mappings), as defined in Section 3.1. However, to simplify the presentation, we only show a simplified variant that uses a quadratic number of auxiliary atoms for comparison, instead of encoding orderings. We therefore use for every two distinct ground atoms $p(D)$, $p'(D')$ of Π , an additional auxiliary predicate $[p(D) \prec p'(D')]$ responsible for storing precedence in the order of derivation. Then, given $\mathcal{R}$ of Fig. 2, for normal programs we just need to add those rules of Fig. 7, yielding $\mathcal{R}''$ shown in Fig. 7. Intuitively, Rules $(\mathcal{R}''12)$ determine precedence among different atoms and Rules $(\mathcal{R}''13)$ take care of transitivity of " $\prec$ ". In addition to Rules $(\mathcal{R}9)$ and $(\mathcal{R}10)$, Rules $(\mathcal{R}''14)$ add a further case of unfoundedness, if precedence among atoms is not suitable for founding. Notice that, we chose to extend $\mathcal{R}$ for simplicity, though the same approach can be applied to $\mathcal{R}'$ by suitably adapting Rules $(\mathcal{R}''14)$.

Example 5.1. Consider the non-ground, normal program $\Pi_3 := \Pi \cup \{r_3\}$ with $r_3 = c(x, y) \leftarrow a(x, y)$, and program Π and facts $\mathcal{F}(\Pi) = \{b(1), c(1, 2), c(2, 1)\}$ from Example 4.1. While the grounding procedure $\mathcal{R}$ can be used identically for normal programs, the additional rules, shown in Fig. 8 (Additional Rules for Foundedness of Normal Programs) ensure orderings as described above. For storing the order of derivation an auxiliary predicate p is used. Notice that the number of rules was kept minimal by using only the relevant variables per rule, i.e., x, y, z for r and x', y' for r_3 , and taking the domain of each variable into consideration. The answer sets of $P_3 = \mathcal{R}''(\Pi_3)$ restricted to predicate symbols a, b and c of Π_3 yield $\{a(1, 1), a(1, 2), b(1), c(1, 1), c(1, 2), c(2, 1)\}$, as expected.

Analogously to Theorem 4.1, one can show correctness of $\mathcal{R}''$ for non-ground, normal programs, including when applied to program parts as in Corollary 4.1. To this end, we capture sufficient conditions of predicate dependencies as follows.

Definition 5.1. (Strongly-Connected Component) Based on the dependency graph $\mathcal{D}_\Pi$ of a non-ground program Π (see Definition 3.4), a set $C \subseteq \text{hpreds}(\Pi)$ of predicates is a *strongly-connected component* (SCC) if C is a $\subseteq$ -maximal set such that for every two distinct predicates p, q in C there is a directed path from p to q in $\mathcal{D}_\Pi$.

Based on that definition, SCCs of the dependency graph can be utilized for partially applying our reduction $\mathcal{R}''$.

Theorem 5.1. (Partial Reducibility/Normal Programs) Let Π be a non-ground, normal program and let Π_1, Π_2 be a partition of Π into programs with $\text{hpreds}(\Pi_1) \cap \text{hpreds}(\Pi_2) = \emptyset$, where for every SCC C_1 of $\mathcal{D}_{\Pi_1}$ and SCC C_2 of $\mathcal{D}_{\Pi_2}$ we have $C_1 \cap C_2 = \emptyset$.

Then, (1) for every answer set M of $\mathcal{R}''(\Pi_1) \cup \mathcal{G}(\Pi_2)$, there is an answer set M' of $\mathcal{R}''(\Pi_1) \cup \mathcal{G}(\Pi_2)$ with $M = M' \cap \text{at}(\mathcal{G}(\Pi))$. Further, (2) for every answer set M' of $\mathcal{R}''(\Pi_1) \cup \mathcal{G}(\Pi_2)$, $M' \cap \text{at}(\mathcal{G}(\Pi))$ is an answer set of $\mathcal{G}(\Pi)$.

$$\begin{aligned}
& p_{ac}(1, 1) \vee p_{ca}(1, 1, 1). \quad p_{ac}(1, 1) \vee p_{ca}(1, 1, 2). & \text{By } (\mathcal{R}''12) \\
& p_{ac}(1, 2) \vee p_{ca}(1, 2, 1). \quad p_{ac}(1, 2) \vee p_{ca}(1, 2, 2). \\
& \text{uf}_r(1, 1) \leftarrow \neg p_{ca}(1, 1, 1), \text{gru}_z(1, 1, 1). & \text{By } (\mathcal{R}''14) \\
& \text{uf}_r(1, 1) \leftarrow \neg p_{ca}(1, 1, 2), \text{gru}_z(1, 1, 2). \\
& \text{uf}_r(1, 2) \leftarrow \neg p_{ca}(1, 2, 1), \text{gru}_z(1, 2, 1). \\
& \text{uf}_r(1, 2) \leftarrow \neg p_{ca}(1, 2, 2), \text{gru}_z(1, 2, 2). \\
& \text{uf}_{r3}(1, 1) \leftarrow \neg p_{ac}(1, 1). \quad \text{uf}_{r3}(1, 2) \leftarrow \neg p_{ac}(1, 2).
\end{aligned}$$

Fig. 8. Additional rules of Fig. 7 for program Π_3 of Example 5.1.

Proof. We prove Statements (1) and (2) independently. For the ease of presentation and without loss of generality, we assume that for every rule $r \in \Pi$ and $h(X) \in H_r$, there is *at least one variable* $y \in \text{var}(r)$ that is not in X , which can be easily achieved by adding a fresh fixed dummy variable to some body predicate.

(1): Let M be an answer set of $\mathcal{G}(\Pi)$ and assume towards a contradiction that there is no extension $M' \supseteq M$ of M with $M \cap \text{at}(\mathcal{G}(\Pi)) = M' \cap \text{at}(\mathcal{G}(\Pi))$ such that M' is an answer set of $\mathcal{R}''(\Pi_1) \cup \mathcal{G}(\Pi_2)$.

Since M is an answer set of $\mathcal{G}(\Pi)$, there is an order $\varphi : M \rightarrow \{0, \dots, |M| - 1\}$ that is used for showing foundedness of M .

Similar to the proof of Theorem 4.1, we collect those head atoms not in M by $N := \{\bar{h}(D) \mid h(X) \in \text{heads}(\Pi_1), D \in \text{dom}(X), h(D) \notin M\}$.

Then, we gather satisfied rules by $S := \{\text{sat}, \text{sat}_r, \text{gr}_x(d) \mid r \in \Pi_1, x \in \text{var}(r), d \in \text{dom}(x)\}$.

We set appropriate domain values for the founded atoms as follows (compare to the proof of Theorem 4.1):

$$\begin{aligned}
F &:= \{\text{gru}_{y_1}(\langle D, D'_{y_1} \rangle), \dots, \text{gru}_{y_\ell}(\langle D, D'_{y_\ell} \rangle) \mid r \in \Pi, h(X) \in H_r, \\
& D' \in \text{dom}(\langle \text{var}(r) \rangle), D = D'_X, \\
M &\supseteq \{h(D)\} \cup \{p(D'_Z) \mid p(Z) \in B_r^+\}, \\
& \{p(D'_Z) \mid p(Z) \in B_r^+, [\varphi(p(D'_Z)) \geq \varphi(h(D))]\} = \emptyset, \\
M \cap \{p(D'_Z) \mid p(Z) \in (B_r^- \cup H_r), p(Z) \neq h(X)\} &= \emptyset, \\
\{y_1, \dots, y_\ell\} &= \{y \in \text{var}(r) \mid y \notin X\}.
\end{aligned}$$

Then, we set domain values for unfounded atoms $U := \{\text{uf}_r(D), \text{gru}_y(\langle D, d(y) \rangle) \mid r \in \Pi_1, h(X) \in H_r, h(D) \in M, y \in \text{var}(r), y \notin X, \text{ there is no } \text{gru}_y(\langle D, d' \rangle) \in F\}$, where $d(y)$ yields any arbitrary, fixed domain value in $\text{dom}(y)$.

We encode ordering φ by setting $O := \{[a < b] \mid \{a, b\} \subseteq M, \varphi(a) < \varphi(b)\}$ and adding those over atoms not in M by $P := \{[p(D) < q(D')] \mid \{p(X), q(Y)\} \subseteq \text{heads}(\Pi_1), p(D) \in \text{dom}(X), q(D') \in \text{dom}(Y), q(D') > p(D), |M \cap \{p(D), q(D')\}| \leq 1\}$, where $>$ is any arbitrary total ordering over atoms $\text{at}(\mathcal{G}(\Pi_1))$.

Finally, we let $M' := M \cup N \cup F \cup U \cup O \cup P$. Since by assumption M' is not an answer set, either (i) one of the rules is not satisfied, or (ii) M' is not minimal.

We proceed by case distinction. Case (i): Some rule $r \in \mathcal{R}''(\Pi_1) \cup \mathcal{G}(\Pi_2)$ is not satisfied by M' . Obviously $\mathcal{G}(\Pi_2)$ is satisfied since M is an answer set of $\mathcal{G}(\Pi)$. Further, it is easy to see that all heads of Rules (R1)–(R8) are satisfied by construction of M' . Rules (R8) are satisfied by construction of F and U . The construction of U also ensures that Rules (R9) and (R10) are satisfied.

Observe that Rules (R11) are satisfied, since M is an answer set of $\mathcal{G}(\Pi_1)$ and therefore every atom of M is founded, as constructed by F .

Further, by construction of O and P , Rules (R''12), (R''13), and (R''14) are satisfied.

This concludes Case (i), since every rule of $\mathcal{R}''(\Pi_1) \cup \mathcal{G}(\Pi_2)$ is satisfied.

Case (ii): Model M' of $\mathcal{R}''(\Pi_1) \cup \mathcal{G}(\Pi_2)$ is not minimal, i.e., there exists a model $M'' \subsetneq M'$ with M'' being a model of $(\mathcal{R}''(\Pi_1) \cup \mathcal{G}(\Pi_2))^{M'}$.

Note that the difference between M' and M'' cannot be due to Rule (R1), since then M' and M'' would be incomparable.

If $\text{sat} \notin M''$, then by Rules (R5), there is a rule $r_i \in \Pi$ with $\text{sat}_{r_i} \notin M''$, which by Rules (R3) and (R4) implies that $\mathcal{G}(\{r_i\})$ is not satisfied by M'' . This, however, contradicts the assumption that M is an answer set of $\mathcal{G}(\Pi)$.

Consequently, by Rules (R6), the difference between M' and M'' cannot be due to any gr_x predicate.

Further, M' and M'' cannot differ in any predicate of the form gru_y , either, since by Rules (R8), for every $h(D) \in M'$, there is precisely one $\text{gru}_y(\langle D, d \rangle) \in M'$.

Similarly, Rules (R9)–(R11) yield deterministic consequences, depending only on the choice of Rules (R1) and (R8).

Rules (R''12) cannot be responsible for the difference between M' and M'' , since then M' and M'' would be incomparable (every answer set has to contain precisely one of these head atoms per grounding of Rules (R''13)). Finally, Rules (R''14) also yield deterministic consequences, given the choice of Rules (R8) and (R''14).

As a result, we conclude that M' and M'' can only differ due to Π_2 . Then, however, $M'' \cap \text{at}(\mathcal{G}(\Pi))$ is also a model of $\mathcal{G}(\Pi_2)^{M'}$. Further, $M'' \cap \text{at}(\mathcal{G}(\Pi))$ is a model of $\mathcal{G}(\Pi)^{M'}$ as well, since M'' is model of $(\mathcal{R}''(\Pi_1) \cup \mathcal{G}(\Pi_2))^{M'}$, which by $\text{hpreds}(\Pi_1) \cap \text{hpreds}(\Pi_2) = \emptyset$, could only be prevented by atoms over body predicates of Π_1 .

Then, however, M is not an answer set since $M'' \cap \text{at}(\mathcal{G}(\Pi)) \subsetneq M$.

(2): Let M' be an answer set of $\mathcal{R}''(\Pi_1) \cup \mathcal{G}(\Pi_2)$ and assume towards a contradiction that $M := M' \cap \text{at}(\mathcal{G}(\Pi))$ is not an answer set of $\mathcal{G}(\Pi)$.

Then, either (i) M is not a model of $\mathcal{G}(\Pi)$, or (ii) there is an $h(D) \in M$ that is unfounded by M . We proceed by case distinction.

Case (i): M is not a model of $\mathcal{G}(\Pi)$. Assume towards a contradiction that there is a rule $r_i \in \Pi$ with M not satisfying $\mathcal{G}(\{r_i\})$. Then, $r_i \in \Pi_1$ since M is by definition a model of $\mathcal{G}(\Pi_2)$. Then, there is a subset $M'' \subsetneq M'$ with $\text{sat}_{r_i} \notin M''$, $\text{sat} \notin M''$ and gr_x predicate set accordingly for $x \in \text{var}(r_i)$ that is a model of $\mathcal{G}(\Pi)$, which contradicts that M' is an answer set (a $\subseteq$ -minimal model of the reduct) of $\mathcal{R}''(\Pi_1) \cup \mathcal{G}(\Pi_2)$.

Case (ii): There exists an $h(D) \in M$ that is unfounded by $\mathcal{R}''(\Pi_1) \cup \mathcal{G}(\Pi_2)$.

If $h \in \text{hpreds}(\Pi_2)$, then by construction of M , $h(D) \in M'$ is also unfounded by every rule in Π_2 (as well as by every rule in Π_1 since $\text{hpreds}(\Pi_1) \cap \text{hpreds}(\Pi_2) = \emptyset$ and there is no SCC overlap that could cause foundedness), contradicting that M' is an answer set.

Otherwise, irrelevant how the predicates gr_x or predicates of the form $\cdot < \cdot$ are assigned in M' , since $h(D) \in M$ is not founded, there is no rule $r_i \in \Pi_1$ that can be grounded such that the resulting body is satisfied and the ground rule founds $h(D)$.

Consequently, for every such rule r_i , by Rules (R9), (R10), and (R''14) there is at least one atom over predicates uf_{r_i} in M' .

Finally, Rules (R11) are not satisfied (for $h(D)$), which contradicts that M' is an answer set of $\mathcal{R}''(\Pi_1) \cup \mathcal{G}(\Pi_2)$. $\square$

In comparison to the procedure $\mathcal{R}'$, which can be computed in time $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^{2-a})$, we see from the runtime complexity that the reduction $\mathcal{R}''$ still can be computed in polynomial time, but might suffer from the ordering encodings.

Theorem 5.2 (Polynomial Runtime and Grounding Size). *Let Π be any non-ground, normal program, where every predicate has arity at most a . Then, the grounding procedure $\mathcal{R}''$ on Π is polynomial, i.e., can be computed in time $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^a)^3$.*

Proof. The reduction $\mathcal{R}''$ is constructed similarly to reduction $\mathcal{R}$, for which we have shown a runtime of $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^{2-a})$. The additional steps of the reduction construct $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^a)^2$ many Rules (R''12) of constant size, $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^a)^3$ many Rules (R''13) of constant size and $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^2 \cdot a)$ many Rules (R''14) of size $\mathcal{O}(a)$. $\square$

5.2. Body-decoupled grounding for horn ASP

Since Horn programs are in fact a subset of normal logic programs, the reduction $\mathcal{R}''$ can be used to compute answer sets of Horn program as well (using the full power of ASP, i.e., Σ_2^P -completeness). However, due to the limitations of Horn logic programs, where only positive normal rules are allowed, there is at most one answer set. Without loss of generality, we assume that a given program *includes at least one constraint*⁴. Notice that, further, since there are only positive rules, we can actually omit ensuring the satisfiability of rules and therefore reduce the complexity of the yielded output. In this way, we can introduce two separate grounding procedures (of lower complexity) to check for consistency and proving atom-subsets being part of an existing answer set.

First, we introduce the grounding procedure $\mathcal{R}_1$ used to decide the inconsistency of a non-ground Horn logic program Π . To this end, the reduction relies on parts of the earlier introduced reduction $\mathcal{R}''$ to (1) guess answer set candidates, (2) prevent foundedness and (3) guess an ordering of derivation. Since Horn programs are only inconsistent if a constraint's body resolves to true, reduction $\mathcal{R}_1$ uses the auxiliary atom uf_i to indicate (4) unfoundedness of a constraint which is derived for every translated constraint. This yields the translation shown in Fig. 9. Given steps (1)–(3) which are applied as in reduction procedure $\mathcal{R}''$, Rules (R''15) capture (4) the unfoundedness of a constraint body through the auxiliary atom uf_i . Then, Rule (R''16) takes care of removing any answer set candidates where the unfoundedness of all constraint bodies is guaranteed. Intuitively, the remaining answer set(s) depict a witness for constraint bodies evaluating to true, giving evidence for the inconsistency of the input program. Naturally, this results can be interpreted and inverted to argue about the consistency of the program. Note that in Rule (R''16), n might be zero, which immediately causes inconsistency if no constraints are contained in Π . Observe that this procedure is limited to Horn programs and in general the unconstrained use of normal constraints containing negation through Rules (R''15) is indeed *not possible*. Intuitively, the reason for this is that for the reduction to correctly decide inconsistency, we require for consistent programs Π that not even a subset of an answer set of $\mathcal{G}(\Pi)$ can be extended to an answer set of the constructed ground program.

Based on this idea, one can construct a procedure $\mathcal{R}_F(\Pi, A)$ to prove the foundation of a subset A of ground atoms within the answer set of a program Π . Fig. 10 shows the new part of the procedure, which replaces Rules (R''15) and (R''16) of Fig. 9 by Rule (R''17). Again, relying on steps (1)–(3) of $\mathcal{R}''$, one can choose a set of atoms which is added to the encoding to decide the foundedness of it in an existing answer set. By adding the Rules (R''17) to $\mathcal{R}''$, the constraints invalidate any guess of the atoms encoded in (1) where the atoms do not hold to remain with answer set candidates including the desired atoms. By construction, procedure $\mathcal{R}_F$ can then be used to construct an encoding for which the consistency problem decides the inclusion of atoms in the *potential* answer set of the input Horn program.

Example 5.2. Consider the non-ground, normal (and Horn) program $\Pi_4 := \Pi_3 \cup \{\leftarrow a(1,1)\}$ with Π_3 and facts $\mathcal{F}(\Pi) = \{b(1), c(1,2), c(2,1)\}$ from Example 5.1. The answer sets for the output of grounding procedure $\mathcal{R}_1$ with additional rules for constraints (see Fig. 11) yield two models, indicating inconsistency of Π_4 , as expected.

Consider the set $A := \{a(1,1), c(1,2), c(2,1)\}$ of ground atoms of Π_3 . The grounding procedure $\mathcal{R}_F$ can be used to decide the membership of A in the answer set of Π_3 . The answer sets of $\mathcal{R}_F(\Pi_3, A)$, i.e., Rules (R''1), (R''8)–(R''14) and negative constraints for each atom in A , yield $\{b(1), a(1,1), a(1,2), c(1,2), c(2,1)\}$ and $\{b(1), a(1,1), a(1,2), c(1,1), c(1,2), c(2,1)\}$ indicating the presence of A in the answer set of Π_3 .

⁴ This is not a hard restriction that could be circumvented by introducing a constraint with an independent atom.

Guess Answer Set Candidates

for every $h(X) \in \text{heads}(\Pi)$, $D \in \text{dom}(X)$:

$$h(D) \vee \bar{h}(D) \leftarrow \quad (\mathcal{R}1)$$

Prevent Unfoundedness

for every $r \in \Pi$, $h(X) \in H_r$, $D \in \text{dom}(X)$, $y \in \text{var}(r)$, $y \notin X$:

$$\bigvee_{d \in \text{dom}(y)} \text{gru}_y(\langle D, d \rangle) \leftarrow h(D) \quad (\mathcal{R}8)$$

for every $r \in \Pi$, $h(X) \in H_r$, $p(Y) \in B_r^+$, $D \in \text{dom}(\langle X, Y \rangle)$, $Y \setminus X = \langle y_1, \dots, y_\ell \rangle$:

$$\text{uf}_r(D_X) \leftarrow \text{gru}_{y_1}(D_{\langle X, y_1 \rangle}), \dots, \text{gru}_{y_\ell}(D_{\langle X, y_\ell \rangle}), \neg p(D_Y) \quad (\mathcal{R}9)$$

for every $h(X) \in \text{heads}(\Pi)$, $D \in \text{dom}(X)$, $\{r_1, \dots, r_m\} = \{r \in \Pi \mid h(X) \in H_r\}$:

$$\leftarrow h(D), \text{uf}_{r_1}(D), \dots, \text{uf}_{r_m}(D) \quad (\mathcal{R}11)$$

Additional Rules for Foundedness of Normal Programs

for every $p(X)$, $p'(X') \in \text{heads}(\Pi)$, $D \in \text{dom}(X)$, $D' \in \text{dom}(X')$, $p(D) \neq p'(D')$:

$$[p(D) \prec p'(D')] \vee [p'(D') \prec p(D)] \leftarrow \quad (\mathcal{R}''12)$$

for every $p_1(X_1)$, $p_2(X_2)$, $p_3(X_3) \in \text{heads}(\Pi)$, $D_1 \in \text{dom}(X_1)$, $D_2 \in \text{dom}(X_2)$, $D_3 \in \text{dom}(X_3)$,

$$p_1(D_1) \neq p_2(D_2), p_1(D_1) \neq p_3(D_3), p_2(D_2) \neq p_3(D_3):$$

$$\leftarrow [p_1(D_1) \prec p_2(D_2)], [p_2(D_2) \prec p_3(D_3)], [p_3(D_3) \prec p_1(D_1)] \quad (\mathcal{R}''13)$$

for every $r \in \Pi$, $h(X) \in H_r$, $p(Y) \in B_r^+$, $D \in \text{dom}(\langle X, Y \rangle)$, $Y \setminus X = \langle y_1, \dots, y_\ell \rangle$, $p(D_Y) \notin \mathcal{F}(\Pi)$:

$$\text{uf}_r(D_X) \leftarrow \text{gru}_{y_1}(D_{\langle X, y_1 \rangle}), \dots, \text{gru}_{y_\ell}(D_{\langle X, y_\ell \rangle}), \neg[p(D_Y) \prec h(D_X)] \quad (\mathcal{R}''14)$$

Additional Rules for Checking Inconsistency

for every $r \in \{r \in \Pi \mid H_r = \emptyset\}$, $p(X) \in B_r^+$, $D \in \text{dom}(X)$, $X = \langle x_1, \dots, x_\ell \rangle$:

$$\text{uf}_r \leftarrow \neg p(D) \quad (\mathcal{R}_115)$$

for $\{r_1, \dots, r_m\} = \{r \in \Pi \mid H_r = \emptyset\}$:

$$\leftarrow \text{uf}_{r_1}, \dots, \text{uf}_{r_m} \quad (\mathcal{R}_116)$$

Fig. 9. Rules of the body-decoupled grounding procedure $\mathcal{R}_1$ for a given non-ground, Horn program Π . The procedure contains additional rules for handling constraints that decide the inconsistency of Π .

Additional Rules for Deciding Membership of Atoms

for every $a \in A$:

$$\leftarrow \neg a \quad (\mathcal{R}_F17)$$

Fig. 10. Additional rules of the body-decoupled grounding procedure $\mathcal{R}_p$, used for deciding the membership of atoms A in the answer set of a given non-ground, Horn program Π .

$$\text{uf}_{r_3} \leftarrow \neg a(1, 1). \quad \text{By } (\mathcal{R}_115)$$

$$\leftarrow \text{uf}_{r_3}. \quad \text{By } (\mathcal{R}_116)$$

Fig. 11. Additional rules of Fig. 9 to decide inconsistency of program Π_4 of Example 5.2.

The following result establishes correctness of the reduction $\mathcal{R}_1$.

Theorem 5.3. (Correctness) Let Π be any non-ground, Horn program. Then, (i) the grounding procedure $\mathcal{R}_1$ on Π is correct, i.e., the program $\Pi' = \mathcal{R}_1(\Pi)$ is inconsistent if and only if $\mathcal{G}(\Pi)$ admits an answer set.

Further, (ii) assume that Π is consistent, i.e., $\mathcal{G}(\Pi)$ admits an answer set. Then, for a set $A \subseteq \text{at}(\mathcal{G}(\Pi))$ of atoms, the program $\Pi'' = \mathcal{R}_F(\Pi, A)$ is consistent if and only if the single answer set M of $\mathcal{G}(\Pi)$ contains A , i.e., $M \supseteq A$.

Proof. First, we show Claim (i). “ $\Rightarrow$ ”: Suppose that Π' is inconsistent. Assume towards a contradiction that there is no interpretation that is a model of $\mathcal{G}(\Pi)$. Then, since Π is Horn, there is a unique interpretation $M \subseteq \text{at}(\mathcal{G}(\Pi))$ such that M is an answer set of those rules $\Pi_H = \{r \in \mathcal{G}(\Pi) \mid H_r \neq \emptyset\}$ whose head is non-empty. Indeed, we can easily define $M := \bigcup_{1 \leq i \leq |\text{at}(\mathcal{G}(\Pi))|} M_i$, where $M_0 = \emptyset$ and $M_i := \{a \mid \{a\} = H_r, r \in \Pi_H, B_r \subseteq M_{i-1}\}$ for every $1 \leq i \leq |\text{at}(\mathcal{G}(\Pi))|$. In the literature this construction is sometimes referred to as the immediate consequence operator T_p , see also [1–3]. Then, the only rules that M could possibly violate are constraints, i.e., the rules in $\Pi_C := \mathcal{G}(\Pi) \setminus \Pi_H$. As a result, there is at least one rule $r \in \Pi_C$ such that $M \not\models r$. However, Π' is inconsistent, and the reason for this inconsistency could lie in Rules ($\mathcal{R}_1'16$). By construction, atoms in M are derived from facts in a linear order; hence, Rules ($\mathcal{R}''13$)

and (R11) are not responsible for the inconsistency. This contradicts the assumption that $M \not\models r$, since Rules ($\mathcal{R}''_1$ 16) can only cause inconsistency if uf_r is derived via Rules ($\mathcal{R}''_1$ 15), which requires that $M \models r$. Therefore, we conclude that Π' is consistent, contradicting our assumption.

“ $\Leftarrow$ ”: Suppose that there is a (unique) answer set M of $\mathcal{G}(\Pi)$. Assume towards a contradiction that Π' admitted an answer set M' . But then, since M' is an answer set, M' satisfies Rule ($\mathcal{R}''_1$ 16). As a result, i.e., in order to satisfy this rule, we require at least one rule $r \in \Pi$ with $H_r = \emptyset$ such that $uf_r \notin M'$. The only rule r' containing $uf_r \in H_{r'}$ is among Rules ($\mathcal{R}''_1$ 15). As a result, we follow that $M' \not\models \mathcal{G}(\{r\})$. By construction of $\mathcal{R}_1$, there can not exist any atom $a \in \text{at}(\mathcal{G}(\Pi)) \setminus M$ that is contained in M' . Indeed, since $a \notin M$, and M is unique due to Π being Horn, a can never be founded, i.e., either one of Rules (R11) are not satisfied, or no suitable total ordering can be found, i.e., an instance of Rules ($\mathcal{R}''_1$ 13) fails. As a result, we conclude $M' \cap \text{at}(\mathcal{G}(\Pi)) \subseteq M$. However, in consequence, already some subset $M'' \subseteq M \cap M'$ invalidates $\mathcal{G}(\{r\})$. Therefore, this yields that $M \not\models \mathcal{G}(\{r\})$, because $B_r^- = \emptyset$. Finally, this contradicts the assumption that M is an answer set of $\mathcal{G}(\Pi)$.

Next, we argue about Claim (ii). “ $\Rightarrow$ ”: Suppose that Π'' admits an answer set M . By construction, and since in particular M is a model of Rules (R11) and of Rules ($\mathcal{R}''_1$ 13), i.e., every atom in M is founded, we follow that M is contained in an answer set of $\mathcal{G}$. By Rules ($\mathcal{R}''_1$ 17), $A \subseteq M$. As a result, we also conclude that A must be contained in an answer set of $\mathcal{G}(\Pi)$.

“ $\Leftarrow$ ”: Assume that A is part of an answer set $M \supseteq A$ of $\mathcal{G}(\Pi)$. Then, one can easily extend M to an answer set M' of Π'' by using any level mapping φ proving that M is founded, which has to exist. Indeed, we define $M' := M \cup \{a < b \mid \{a, b\} \subseteq \text{at}(\mathcal{G}(\Pi)), \varphi(a) < \varphi(b)\}$. Clearly, M' is a model of Rules (R11), independent of the choice given by Rules (R8). Further, since φ proves foundedness of M , we conclude that M' is also a model of Rules ($\mathcal{R}''_1$ 13). As a result, M' is an answer set of Π'' . $\square$

Theorem 5.4. (Polynomial Runtime and Grounding Size) Let Π be any non-ground, Horn program, where every predicate has arity at most a . Then, the grounding procedures $\mathcal{R}_1$ and $\mathcal{R}_F$ on Π are polynomial, i.e., can be computed in time $\mathcal{O}((\|\Pi\| \cdot |\text{dom}(\Pi)|)^{a^3})$.

Proof. The reductions $\mathcal{R}_1$ and $\mathcal{R}_F$ are constructed similarly to reduction $\mathcal{R}''$, for which we have shown a runtime of $\mathcal{O}((\|\Pi\| \cdot |\text{dom}(\Pi)|)^{a^3})$. The additional steps of the reductions construct $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^a)$ many Rules ($\mathcal{R}''_1$ 15) of constant size, one Rule ($\mathcal{R}''_1$ 16) of size $\mathcal{O}(\|\Pi\|)$ and $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^a)$ many Rules ($\mathcal{R}''_1$ 17) of constant size. $\square$

5.3. Body-decoupled grounding for disjunctive ASP

Further, the reduction can be pursued beyond normal logic programs, i.e., to disjunctive (or full) logic programs. In comparison to the above introduced reductions, the approach is considering ground ELP programs instead of ASP programs, which is motivated in the following.

While the motivation for body-decoupled grounding still applies, Table 2 shows that the complexity of ground ASP programs does not allow for a similar reduction (as in Sections 4 and 5.1) from disjunctive programs since its complexity is limited to Σ_2^P , whereas (full) non-ground programs are known to be Σ_3^P -complete (when assuming bounded predicate arity). We therefore require a problem of similar complexity for which solvers are available and to which we can (efficiently) reduce from non-ground programs.

As defined in Section 3.3, epistemic logic programs implement a way to go beyond the expressive power of ASP programs. ELP presents an extension to standard ASP and allows reasoning over multiple worlds, i.e., deriving certain consequences depending on whether objections are known or possible. This increase in expressiveness enables us to encode harder problems, as the complexity of (ground) ELP ascends to Σ_3^P -completeness [13]. This lies in consensus with the complexity of non-ground ASP programs (cf. Table 1 ②), which seems to be fitting for our reduction approach. Further, since ELP presents an extension to standard logic programs an efficient reduction from ASP and to ELP seems obvious.

Interestingly, epistemic logic programs were subject to various works in the last years, with a special focus on methods to efficiently solve these programs. In particular, we list approaches using reductions [28], multi-shot solving as an extension to CLINGO [27], and hybrid solving techniques that utilize treewidth to break down the effort of solving [29]. These improvements make ELP an attractive target formalism.

Similar to the earlier reductions, we assume a given non-ground program Π and a set $\mathcal{F}(\Pi)$ of facts. As for the approach for normal programs, we use every instantiation of $p(X)$ and its negation $\neg p(X)$ over $\text{dom}(\Pi)$ for each predicate $p(X)$ in $\text{heads}(\Pi)$, resulting in atoms $\text{AtPred} := \{p(D), \neg p(D) \mid p(X) \in \text{heads}(\Pi), D \in \text{dom}(X)\}$. Further, we define $\text{AtPredC} := \{p^c(X), \neg p^c(X) \mid p(X) \in \text{AtPred}\}$.

In accordance with the semantics of ASP, we require to ensure (i) satisfiability of a potential model M of Π and (ii) non-existence of counter witnesses, i.e., a model $N \subsetneq M$ (satisfying (i)) of Π^M . Notice that by performing subset minimization we do not require a foundedness check as in earlier versions. For (i), the computation of models of rules, we require atoms $\text{AtSat}^1 := \{\text{sat}^1, \text{sat}_x^1, \text{gr}_x^1(d) \mid r \in \Pi, x \in \text{var}(r), d \in \text{dom}(x)\}$, where sat^1 (sat_x^1) indicates satisfiability (of non-ground rule r) for a potential model $1 \in \{\epsilon, c\}$, with ϵ denoting model M and c a potentially smaller model N . An atom of the form $\text{gr}_x^1(d)$ indicates that for checking satisfiability, we assign variable x of non-ground rule r to domain value $d \in \text{dom}(x)$. For (ii), i.e., the non-existence of a model $N \subsetneq M$, we require proper subsets. For this purpose we use $\text{AtEq} := \{\text{eq}_{p(X)} \mid p(X) \in \text{AtPred}\} \cup \{\text{empty}\}$ to intuitively derive the equality of a predicate instantiation for the potential models M and N while considering empty candidates.

Our reduction $\mathcal{R}$ consists of the following parts that transforms a disjunctive, non-ground program Π into an epistemic, ground program $\mathcal{R}(\Pi)$ consisting of $\mathcal{F}(\Pi)$ and the rules given in Fig. 13. To generate all possible world view candidates, our targeted models M , we use Rules (R1) and (R2), where we use the epistemic negation to generate the mutually exclusive assignments (through atoms a and $\bar{a}$) at the outermost level (ELP). For (ii), we guess possible smaller models N throughout these candidates, achieved by Rules (R3), which results (for each world view candidate) in an answer set candidate for all possible subsets of the guessed world view candidates. Then, Rules (R4)–(R7) trivially derive the equality of the guessed head predicates ensuring that the guessed models N only contain

Ensure Satisfiability of Answer-Set Candidates

for every $r \in \Pi$, $x \in \text{var}(r)$:

$$\bigvee_{d \in \text{dom}(x)} \text{gr}_x(d) \leftarrow \quad (\text{S1})$$

for every $r \in \Pi$, $p(X) \in B_r^+$, $D \in \text{dom}(X)$, $X = \langle x_1, \dots, x_\ell \rangle$:

$$\text{sat}_r \leftarrow \text{gr}_{x_1}(D_{\langle x_1 \rangle}), \dots, \text{gr}_{x_\ell}(D_{\langle x_\ell \rangle}), \neg p(D) \quad (\text{S2})$$

for every $r \in \Pi$, $p(X) \in H_r \cup B_r^-$, $D \in \text{dom}(X)$, $X = \langle x_1, \dots, x_\ell \rangle$:

$$\text{sat}_r \leftarrow \text{gr}_{x_1}(D_{\langle x_1 \rangle}), \dots, \text{gr}_{x_\ell}(D_{\langle x_\ell \rangle}), p(D) \quad (\text{S3})$$

where $\Pi = \{r_1, \dots, r_n\}$:

$$\text{sat} \leftarrow \text{sat}_{r_1}, \dots, \text{sat}_{r_n} \quad (\text{S4})$$

for every $r \in \Pi$, $x \in \text{var}(r)$, $d \in \text{dom}(x)$:

$$\text{gr}_x(d) \leftarrow \text{sat} \quad (\text{S5})$$

Ensure Satisfiability of $\subseteq$ -Smaller Models

for every $r \in \Pi$, $x \in \text{var}(r)$:

$$\bigvee_{d \in \text{dom}(x)} \text{gr}_x^c(d) \leftarrow \quad (\text{S6})$$

for every $r \in \Pi$, $p(X) \in B_r^+$, $D \in \text{dom}(X)$, $X = \langle x_1, \dots, x_\ell \rangle$:

$$\text{sat}_r^c \leftarrow \text{gr}_{x_1}^c(D_{\langle x_1 \rangle}), \dots, \text{gr}_{x_\ell}^c(D_{\langle x_\ell \rangle}), \neg p^c(D) \quad (\text{S7})$$

for every $r \in \Pi$, $p(X) \in H_r$, $D \in \text{dom}(X)$, $X = \langle x_1, \dots, x_\ell \rangle$:

$$\text{sat}_r^c \leftarrow \text{gr}_{x_1}^c(D_{\langle x_1 \rangle}), \dots, \text{gr}_{x_\ell}^c(D_{\langle x_\ell \rangle}), p^c(D) \quad (\text{S8})$$

for every $r \in \Pi$, $p(X) \in B_r^-$, $D \in \text{dom}(X)$, $X = \langle x_1, \dots, x_\ell \rangle$:

$$\text{sat}_r^c \leftarrow \text{gr}_{x_1}^c(D_{\langle x_1 \rangle}), \dots, \text{gr}_{x_\ell}^c(D_{\langle x_\ell \rangle}), p(D) \quad (\text{S9})$$

where $\Pi = \{r_1, \dots, r_n\}$:

$$\text{sat}^c \leftarrow \text{nempty}, \text{sat}_{r_1}^c, \dots, \text{sat}_{r_n}^c \quad (\text{S10})$$

for every $r \in \Pi$, $x \in \text{var}(r)$, $d \in \text{dom}(x)$:

$$\text{gr}_x^c(d) \leftarrow \text{sat}^c \quad (\text{S11})$$

Fig. 12. Reduction $S(\Pi)$ for encoding satisfiability of a non-ground program Π into a ground program requiring the use of disjunction through Formulas (S1) and (S6).

proper subsets of models M , where we use Rules (R6) to consider and not remove the empty set candidate (for which no proper subset exists and the construction would result in removing it). As in earlier reductions, see Fig. 12, we ensure (i) satisfiability of models M by Rules (S1)–(S5), where we yield sat_r whenever there is an assignment of variables to domain values for a non-ground rule $r \in \Pi$. If such an atom can be derived for all non-ground rules $r \in \Pi$, we follow sat by Rule (S4), which is mandatory throughout the world view (cf. Rule (R8); an epistemic constraint to require atom sat to hold within a world view, i.e., for every answer set of the epistemic reduct sat holds). Rule (S5) applies *saturation*, causing the assignment of all domain values to every variable. In this way, any world view candidate M is removed if not complying with (i). In the same way, we ensure (ii) satisfiability for the potential models N (proper subsets of M), removing every answer set candidate (within each world view candidate) that does not conform with (i). Notice that, in addition to Rules (S6)–(S8) and (S11), which are copying the behavior of Rules (S1)–(S3) and (S5), we use Rules (S9) to “satisfy” rules of Π that are removed by the construction of the GL reduct Π^M and instead of Rules (S4), we use Rules (S10) to consider a potentially empty model for which we do not need to check for satisfiability of (non-existing) proper subsets. Then, Rule (R9) removes any world view candidate M , where there is an answer set candidate deriving sat^c such that we ensure (ii) non-existence of a model $N \subsetneq M$ of Π^M . This is encoded through a constraint for the epistemic negation of the negated atom sat^c , which ensures that none of the answer sets of the epistemic reduct includes the atom sat^c . Intuitively, the only world view candidates remaining depict models of Π , where none of the answer set candidates contains sat^c , i.e., there is no proper subset N of the guessed model M (represented by the world view candidate) which represents a model of the GL reduct Π^M .

Notice that, in comparison to the earlier presented reductions, the rules for guaranteeing foundedness of guessed head atoms and ordering are not required anymore in our reduction R. This is due to the fact that the reduction encodes subset-minimization (cf. Definition of an Answer Set, see Definition 3.2), which implicitly guarantees the other properties.

Example 5.3. Consider the non-ground, disjunctive program $\Pi_5 := \{a(x, y) \vee p(x, y) \leftarrow b(x), c(y, z)\}$ and facts $F(\Pi_5) := \{b(1), c(1, 2), c(2, 1)\}$. Grounding the program as described above and shown in Fig. 13, results in the ground program $P_5 = F(\Pi_5) \cup R(\Pi_5)$ of Figs. 14 and 15. The world view candidates that are guessed by the resulting ground program are checked by the procedure described above, resulting in the four world views $\{a(1, 1), a(1, 2), b(1), c(1, 2), c(2, 1)\}$, $\{a(1, 1), p(1, 2), b(1), c(1, 2), c(2, 1)\}$,

Guess Answer-Set Candidatesfor every $h(X) \in \text{heads}(\Pi)$, $D \in \text{dom}(X)$, $a = h(D)$:

$$a \leftarrow \text{not } \dot{a} \quad (\text{R1})$$

for every $h(X) \in \text{heads}(\Pi)$, $D \in \text{dom}(X)$, $a = h(D)$:

$$\dot{a} \leftarrow \text{not } a \quad (\text{R2})$$

Guess Potentially $\subseteq$ -Smaller Modelsfor every $h(X) \in \text{heads}(\Pi)$, $D \in \text{dom}(X)$, $a = h(D)$:

$$a^c \vee \dot{a}^c \leftarrow a \quad (\text{R3})$$

Prevent Spuriously $\subseteq$ -Smaller Modelsfor every $h(X) \in \text{heads}(\Pi)$, $D \in \text{dom}(X)$, $a = h(D)$:

$$\text{eq}_a \leftarrow a^c \quad (\text{R4})$$

for every $h(X) \in \text{heads}(\Pi)$, $D \in \text{dom}(X)$, $a = h(D)$:

$$\text{eq}_a \leftarrow \neg a \quad (\text{R5})$$

for every $h(X) \in \text{heads}(\Pi)$, $D \in \text{dom}(X)$, $a = h(D)$:

$$\text{nempty} \leftarrow a \quad (\text{R6})$$

for every $h(X) \in \text{heads}(\Pi)$, $\{a_1, \dots, a_n\} = \{h(D) \mid D \in \text{dom}(X)\}$, $1 \leq i \leq n$:

$$\leftarrow \text{nempty}, \text{eq}_{a_1}, \dots, \text{eq}_{a_n} \quad (\text{R7})$$

Prevent Unsatisfied Rules and $\subseteq$ -Smaller Models

$$\leftarrow \text{not sat} \quad (\text{R8})$$

$$\leftarrow \text{not } \neg \text{sat}^c \quad (\text{R9})$$

Ensure Satisfiability of Non-Ground Rules: see Figure 12**Fig. 13.** Reduction $R(\Pi)$ from a disjunctive, non-ground program Π to an epistemic, ground program.**Ensure Satisfiability of a potential witness M**

$$\text{gr}_x(1). \text{gr}_y(1) \vee \text{gr}_y(2). \text{gr}_z(1) \vee \text{gr}_z(2). \quad \text{By (S1)}$$

$$\text{sat}_r \leftarrow \text{gr}_x(1), \neg b(1). \quad \text{By (S2)}$$

$$\text{sat}_r \leftarrow \text{gr}_y(1), \text{gr}_z(1), \neg c(1, 1). \text{sat}_r \leftarrow \text{gr}_y(1), \text{gr}_z(2), \neg c(1, 2).$$

$$\text{sat}_r \leftarrow \text{gr}_y(2), \text{gr}_z(1), \neg c(2, 1). \text{sat}_r \leftarrow \text{gr}_y(2), \text{gr}_z(2), \neg c(2, 2).$$

$$\text{sat}_r \leftarrow \text{gr}_x(1), \text{gr}_y(1), a(1, 1). \text{sat}_r \leftarrow \text{gr}_x(1), \text{gr}_y(2), a(1, 2). \quad \text{By (S3)}$$

$$\text{sat}_r \leftarrow \text{gr}_x(1), \text{gr}_y(1), p(1, 1). \text{sat}_r \leftarrow \text{gr}_x(1), \text{gr}_y(2), p(1, 2)$$

$$\text{sat} \leftarrow \text{sat}_r. \quad \text{By (S4)}$$

$$\text{gr}_x(1) \leftarrow \text{sat}. \text{gr}_y(1) \leftarrow \text{sat}. \text{gr}_y(2) \leftarrow \text{sat}. \quad \text{By (S5)}$$

$$\text{gr}_z(1) \leftarrow \text{sat}. \text{gr}_z(2) \leftarrow \text{sat}.$$

Ensure Satisfiability of a potential counterwitness C

$$\text{gr}_x^c(1). \text{gr}_y^c(1) \vee \text{gr}_y^c(2). \text{gr}_z^c(1) \vee \text{gr}_z^c(2). \quad \text{By (S6)}$$

$$\text{sat}_r^c \leftarrow \text{gr}_x^c(1), \neg b^c(1). \quad \text{By (S7)}$$

$$\text{sat}_r^c \leftarrow \text{gr}_y^c(1), \text{gr}_z^c(1), \neg c^c(1, 1). \text{sat}_r^c \leftarrow \text{sat}_y^c(1), \text{gr}_z^c(2), \neg c^c(1, 2).$$

$$\text{sat}_r^c \leftarrow \text{gr}_y^c(2), \text{gr}_z^c(1), \neg c^c(2, 1). \text{sat}_r^c \leftarrow \text{gr}_y^c(2), \text{gr}_z^c(2), \neg c^c(2, 2).$$

$$\text{sat}_r^c \leftarrow \text{gr}_x^c(1), \text{gr}_y^c(1), a^c(1, 1). \text{sat}_r^c \leftarrow \text{gr}_x^c(1), \text{gr}_y^c(2), a^c(1, 2). \quad \text{By (S8)}$$

$$\text{sat}_r^c \leftarrow \text{gr}_x^c(1), \text{gr}_y^c(1), p^c(1, 1). \text{sat}_r^c \leftarrow \text{gr}_x^c(1), \text{gr}_y^c(2), p^c(1, 2)$$

$$\text{sat}^c \leftarrow \text{sat}_r^c, \text{nempty}. \quad \text{By (S10)}$$

$$\text{gr}_x^c(1) \leftarrow \text{sat}^c. \text{gr}_y^c(1) \leftarrow \text{sat}^c. \text{gr}_y^c(2) \leftarrow \text{sat}^c. \quad \text{By (S11)}$$

$$\text{gr}_z^c(1) \leftarrow \text{sat}^c. \text{gr}_z^c(2) \leftarrow \text{sat}^c.$$

Fig. 14. $S(\Pi_5)$ with Π_5 from Example 5.3, guided by R of Fig. 12.

Guess Answer-Set Candidates

$$a(1, 1) \leftarrow \text{not } \dot{a}(1, 1). \quad a(1, 2) \leftarrow \text{not } \dot{a}(1, 2). \quad \text{By (R1)}$$

$$p(1, 1) \leftarrow \text{not } \dot{p}(1, 1). \quad p(1, 2) \leftarrow \text{not } \dot{p}(1, 2).$$

$$\dot{a}(1, 1) \leftarrow \text{not } a(1, 1). \quad \dot{a}(1, 2) \leftarrow \text{not } a(1, 2). \quad \text{By (R2)}$$

$$\dot{p}(1, 1) \leftarrow \text{not } p(1, 1). \quad \dot{p}(1, 2) \leftarrow \text{not } p(1, 2).$$

Guess Potentially $\subseteq$ -Smaller Models

$$a^c(1, 1) \vee \dot{a}^c(1, 1) \leftarrow a(1, 1). \quad a^c(1, 2) \vee \dot{a}^c(1, 2) \leftarrow a(1, 2). \quad \text{By (R3)}$$

$$p^c(1, 1) \vee \dot{p}^c(1, 1) \leftarrow p(1, 1). \quad p^c(1, 2) \vee \dot{p}^c(1, 2) \leftarrow p(1, 2).$$

Prevent Spuriously $\subseteq$ -Smaller Models

$$\text{eq}_{a(1,1)} \leftarrow a^c(1, 1). \quad \text{eq}_{a(1,2)} \leftarrow a^c(1, 2). \quad \text{By (R4)}$$

$$\text{eq}_{p(1,1)} \leftarrow p^c(1, 1). \quad \text{eq}_{p(1,2)} \leftarrow p^c(1, 2).$$

$$\text{eq}_{a(1,1)} \leftarrow \neg a(1, 1). \quad \text{eq}_{a(1,2)} \leftarrow \neg a(1, 2). \quad \text{By (R5)}$$

$$\text{eq}_{p(1,1)} \leftarrow \neg p(1, 1). \quad \text{eq}_{p(1,2)} \leftarrow \neg p(1, 2).$$

$$\text{nempty} \leftarrow a(1, 1). \quad \text{nempty} \leftarrow a(1, 2). \quad \text{By (R6)}$$

$$\text{nempty} \leftarrow p(1, 1). \quad \text{nempty} \leftarrow p(1, 2).$$

$$\leftarrow \text{nempty}, \text{eq}_{a(1,1)}, \text{eq}_{a(1,2)}, \text{eq}_{p(1,1)}, \text{eq}_{p(1,2)}. \quad \text{By (R7)}$$

Prevent Unsatisfied Rules and $\subseteq$ -Smaller Models

$$\leftarrow \text{not sat}. \quad \text{By (R8)}$$

$$\leftarrow \text{not } \neg \text{sat}^c. \quad \text{By (R9)}$$

Fig. 15. $R(\Pi_5)$ with Π_5 from Example 5.3, guided by R of Fig. 13.

$\{p(1, 1), a(1, 2), b(1), c(1, 2), c(2, 1)\}$ and $\{p(1, 1), p(1, 2), b(1), c(1, 2), c(2, 1)\}$ (when restricted to the predicate symbols of Π_5) which are equal to the answer sets of the original non-ground program Π_5 .

Similar to the earlier reductions $\mathcal{R}$ and $\mathcal{R}''$ we state correctness of the reduction R in the following.

Theorem 5.5. (Correctness) *Let Π be any disjunctive, non-ground program. Then, the grounding procedure R on Π is correct, i.e., the world views of $R(\Pi)$ restricted to $\text{at}(\mathcal{G}(\Pi))$ match with the answer sets of $\mathcal{G}(\Pi)$.*

Precisely, (a) for every world view W of $R(\Pi)$, there is an answer set M of $\mathcal{G}(\Pi)$, such that $M = W \cap \text{at}(\mathcal{G}(\Pi))$. Further, (b) every answer set M of $\mathcal{G}(\Pi)$ can be extended to a world view of $R(\Pi)$.

Proof. (Sketch) Conceptually, the proof works similarly to the one of Theorem 4.1. The difference here is that we relate world views of $R(\Pi)$ with answer sets of $\mathcal{G}(\Pi)$, which makes the proof technical and more involved. For technical details and further insights, we refer to Appendix A. $\square$

Interestingly, the evaluation of the runtime of reduction R shows that the upper bound is lower than for any other reduction mentioned before. This is due to the fact that reduction R encodes subset minimization and therefore does neither require checking foundedness (cf. Fig. 2) nor derivation orderings (cf. Fig. 7), which depict the most effort of the other reductions.

Theorem 5.6. (Runtime) *Let Π be any disjunctive, non-ground program, where every predicate has arity at most a . Then, the grounding procedure R on Π is polynomial, i.e., can be computed in time $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^a)$.*

Proof. The reduction R constructs $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^a)$ many Rules (R1), (R2) and (R3) of constant size for guessing candidates. For preventing spuriously smaller models $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^a)$ many Rules (R4), (R5) and (R6) of constant size, as well as one Rule (R7) of size $\mathcal{O}(\|\Pi\|)$ are constructed. For the component S, we require: $\mathcal{O}(\|\Pi\| \cdot a)$ many Rules (S1) and (S6) of size $|\text{dom}(\Pi)|$, $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|^a)$ many Rules (S2). Then, (S3), (S7), (S8) and (S9) of size $\mathcal{O}(a)$, one of each Rule (S4) and (S10) of size $\mathcal{O}(\|\Pi\|)$, as well as $\mathcal{O}(\|\Pi\| \cdot |\text{dom}(\Pi)|)$ many Rules (S5) and (S11) of constant size. Further, the reduction R constructs one Rule (R8) and (R9) of constant size. $\square$

6. Implemented prototype and experiments

To show our approach in practical environments we implemented an open-source prototype software tool called NEWGROUND⁵ Accordingly, the system NEWGROUND implements our reduction $\mathcal{R}'$ for non-ground, tight programs to realize body-decoupled grounding via search as described in Section 4.

⁵ The system (including supplemental material) is available at <https://github.com/viktorbesin/newground>.

To this end, we designed NEWGROUND as a reusable module that translates non-ground input programs into ground, disjunctive programs using our reductions. In this way, the system stays independent of any potentially used solver, i.e., the user stays in control of which grounder, solver etc. is used afterwards, making NEWGROUND universally applicable (even for future solvers). For this reason we settled for a command line tool, which is not only basically standard for ASP related software, but also allows for fast piping of inputs and outputs as well as flawlessly cooperating with Python3, which was used for the implementation. Further, we utilize the latest CLINGO Python API for efficiently parsing inputs profiting from CLINGO's longstanding development of efficient ASP software.

The system NEWGROUND implements the optimized reduction R' for non-ground, tight programs, featuring the improved rule set for checking foundedness, which exploits mutual variable independencies, as highlighted in Fig. 5. Furthermore, since we still recognize the strengths of traditional, sophisticated grounding systems, NEWGROUND also supports partial reducibility (cf. Corollary 4.1). This feature allows users to specify which parts of a program should be grounded using our reduction, and which parts should be handled by traditional grounding methods. Rather than enforcing a single strategy, this offers a valuable degree of control for expert users aiming to tackle complex applications – particularly in cases where grounding bottlenecks are known in advance and selective reduction can help avoid infeasible computations.

6.1. Benchmarks

In order to evaluate the performance of NEWGROUND, we designed a series of benchmarks. Clearly, we cannot beat established and highly optimized grounders in all imaginable scenarios and instances. Instead, the goal of our prototype and these benchmark settings is to visualize and discuss the potential and strengths of our approach. Further, we point out use-cases where body-decoupled grounding might be preferable with its advantages over traditional approaches. Indeed, body-decoupled grounding can be incorporated into every grounder, e.g., as part of a portfolio. These benchmark settings are used to discuss potential ways to do so.

6.2. Benchmark scenarios

In order to test NEWGROUND, the following graph-like *scenarios* are considered. In this, we apply the ideas for improvement via partial reducibility of Section 4.2 for tight programs. Note that while some decision problems surrounding these computational scenarios below are polynomial-time decidable, in logic programming these might appear as subquestions or as parts of larger optimization, reasoning, or even counting tasks. Indeed, our goal is to focus on designing a novel grounder for scenarios, where grounding is not possible (or is beyond several GBs). Hence, in these cases, solving is mostly useless due to huge ground programs.

- S1 (coloring):** Compute *edge colorings* over three colors in directed graphs such that for each node no incoming or outgoing edges have the same color.
- S2 (paths):** Find *reachable paths* between source and destination nodes, where each node admits only one outgoing edge.
- S3 (clique):** Obtain directed subgraphs containing so-called *cliques*, i.e., fully connected subgraphs of size at least three.
- S4 (nprc):** Compute non-partition-removal colorings: remove one vertex such that the transitive closure of the original and the resulting graph are equal on the remaining nodes and that the resulting graph is 3-colorable. The encoding is taken from [59].
- S5 (stable marriage):** Obtain so-called *stable marriages*. The encoding is taken from the ASP competition 2014.⁶

The selection of scenarios is justified as follows. With Scenario S1 (coloring) we aim at providing a basic coloring problem. While the decision of Scenario S2 (paths) is in P, the problem can be extended to counting: counting such paths is as hard as every #P problem [107]. Deciding Scenario S3 (clique) is in polynomial-time computable, but ready to be used as part of more expressive problems. Finally, we consider Scenarios S4 (nprc) and S5 (stable marriage) in order to cover known ASP scenarios. Note that both scenarios are quite different. Indeed, S4 (nprc) is from the context of lazy grounding, where the grounding bottleneck is indeed an issue. In contrast, S5 (stable marriage) is from the discontinued ASP competition, which are not designed to suffer from the grounding bottleneck. In fact, the design of the ASP competition instances is such that almost all instances can be grounded within seconds.

Still, one might wonder why we did not study more ASP competition instances. The reason is that for these instances, which are not designed to run into a grounding bottleneck, there is no purpose in punishing our approach by showing that for instances without grounding issues, body-decoupled grounding does not perform particularly well. In fact, the only reason why we designed body-decoupled grounding is to decouple long rule bodies in grounding-heavy scenarios. Recently, we designed a grounding heuristics [108] that enable to quickly decide whether body-decoupled grounding pays off at all.⁷ It turns out that the computation of this heuristics works with only little overhead, so on the discontinued ASP competition instances, mostly standard grounding is selected (within seconds). Then, compared to GRINGO or IDLV, on easy-to-ground instances, one achieves about the same performance for such a procedure comprising the decision heuristics plus GRINGO or IDLV. Such heuristics should make it convenient for incorporating body-decoupled grounding into newer grounder revisions of GRINGO or IDLV.

⁶ The details of the ASP competition 2014 are available at <https://www.mat.unical.it/aspcomp2014>.

⁷ The work for designing such heuristics [108] has been recently presented at ICLP 2025. Note, however, that this is follow-up work, which requires a different skill set, as designing efficient heuristics is a completely different topic than the reduction-based approach inspired by complexity theory that is designed here.

Partial Reductions for NEWGROUND. As body-decoupled grounding is meant as a relief for those rules that hold a crucial role in terms of grounding size (grounding bottleneck), we chose to apply our reduction through NEWGROUND for the following rules.

- S1 (coloring): on the rules prohibiting identical edge colors
- S2 (paths): on the rule restricting the path to 1 chosen outgoing edge per node
- S3 (clique): on the rule checking for a clique
- S4 (nprc): on a rule that ensures non-partition (over reachability)
- S5 (stable marriage): on the rules preventing polygamy

In the encodings, the respective rules below “#program rules.” are the ones subject to reduction and NEWGROUND grounds all other rules via GRINGO or IDLV (NEWGROUND*). For all other benchmarked vanilla solvers, the lines indicating subprograms are removed before those solvers are invoked.

Based on these scenarios, we study corresponding hypotheses that shall be verified in this section.

- H1:** In contrast to traditional grounding, body-decoupled grounding of NEWGROUND suffers less from increased *instance density* and *instance size*.
- H2:** Body-decoupled grounding can massively reduce *grounding sizes* and *grounding times* of large instances.
- H3:** Body-decoupled grounding can *improve overall and solving performance* on crafted and application instances.
- H4:** The idea of body-decoupled grounding, where suitable, efficiently interoperates with other approaches.

6.3. Benchmark instances

To evaluate the hypotheses, we use both crafted (random) instances and suitable ASP competition instances. Note that *competition instances are not particularly designed* to run into grounding bottlenecks. Therefore, we randomly generate instances of (directed) graphs of different sizes ranging from 100 to 1500 vertices with an edge probability (instance density) from 0.1 to 1.0, for S1–S4. Here, instance density refers to the probability that a directed edge exists between any two vertices in the graph. These are particularly useful to test Hypotheses H1–H3. For S5, we took competition instances (as well as crafted ones), which aid in analyzing Hypotheses H2–H4.

6.4. Compared tools

For the evaluation of our tool we only study the performance of the following *exact (full) grounders*, i.e., we do not consider any other modern approaches, e.g., lazy grounding or ASP module theory.

- GRINGO: version 5.5.1
- IDLV: version 1.1.6
- NEWGROUND: where Body-decoupled grounding is applied on certain (manually fixed, see [Section 6.2](#)) non-ground rules of the respective programs that potentially cause grounding overhead. The remaining part is grounded with GRINGO.
- NEWGROUND*: similarly to NEWGROUND with IDLV being used instead of GRINGO.

While evaluating these benchmarks we measure *grounding sizes* and *grounding times* of the mentioned grounders. Further, we measure *solving capabilities* of the resulting ground programs for comparing solving. For full intercomparability, grounding size measures the size of the corresponding text output without writing it to the persistent storage (no disk or I/O operations). Anyway, relative orders of magnitude of measured grounding sizes unambiguously stand and are independent of format optimizations (which could be applied to any grounder).

For comparing the *overall performance* of the resulting ground programs, we use solver CLINGO version 5.5.1 with options `-q --stats=2` to compute one answer set without any excessive output. For solving the resulting ground programs of NEWGROUND we add the option `--project` to ensure that answer sets are over the same atoms. We *limit* the main memory (RAM) of the cluster to 16GB and overall runtimes (grounding & solving) to 1800s. For all resulting plots we *cut-off* grounding sizes beyond 10GB (or 30GB where reasonable).

6.5. Benchmark platform

All of our benchmarks were conducted on a cluster consisting of 12 nodes. Each node of the cluster is equipped with two Intel Xeon E5-2650 CPUs, and each of these 12 physical cores runs at a 2.2GHz clock speed and has access to 256 GB of shared RAM. Results are gathered on a system running Ubuntu 16.04.1 LTS OS that is powered on kernel 4.4.0–139. We disabled hyperthreading and used Python version 3.7.6.

6.6. Results

In the following we discuss the results of the performed experiments. To this end, we evaluate the results in terms of grounding scalability, grounding performance and overall performance to verify our hypothesis H1–H4. Further, where appropriate, we compare to the better performing contestant (grounding profiles) and similarly, we limit the results to the better performing alternative of NEWGROUND (grounding times).

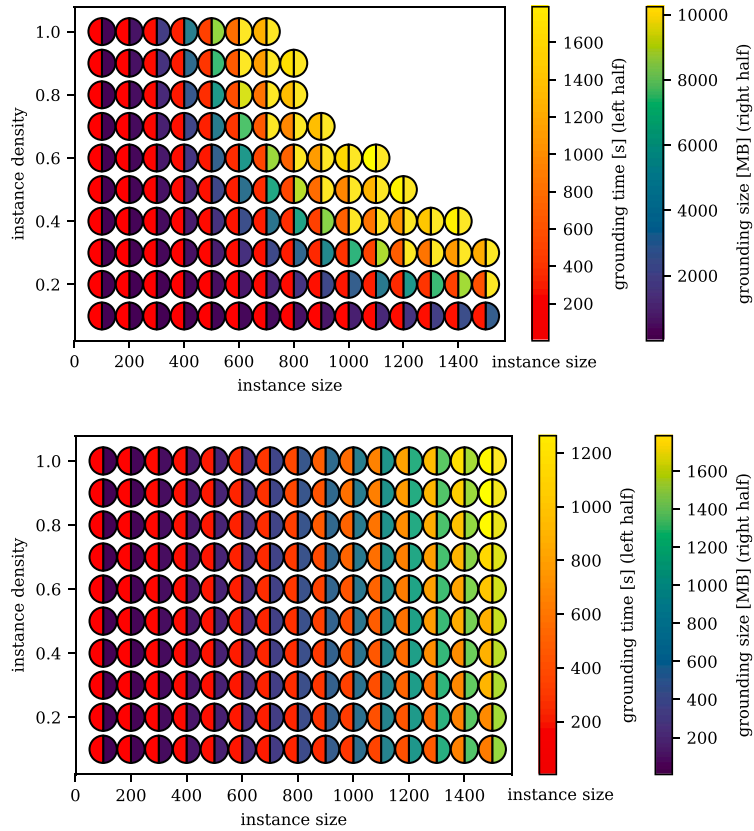


Fig. 16. (Top): Grounding profile for GRINGO of S1 (coloring). The x-axis refers to the instance size and the y-axis indicates the density. The presence of a circle indicates that an instance was grounded below < 1800 s, where the color of the left half depicts the grounding time and the right half depict grounding size. Mind the different scales. (Bottom): Grounding profile for NEWGROUND of S1.

Grounding scalability. For studying and comparing grounded programs yielded by the different tools, we use crafted instances for Scenarios S1–S4 to generate a grounding profile for each tool. Fig. 16 (Top) and 16 (Bottom) present these grounding profiles for the best contestant and NEWGROUND⁸, where grounding times and sizes are shown depending on the instances size (x-axis) and instance density (y-axis). Circles indicate a grounding time of less than 1800s where the color of the left half depicts the grounding time and the right half the grounding size. Notice that different scales are used to allow for a more distinct value read-off throughout the plots.

Throughout the plots it can be seen that, compared to GRINGO (and IDLV), NEWGROUND grounds larger and denser instances faster. Further, one can clearly identify the reduction of grounding sizes of up to $\frac{1}{50}$ (cf. Fig. B.1 (Top) and B.1 (Bottom)). Interestingly, when looking at fixed instance sizes (columns) for NEWGROUND (Figs. 16 (Bottom), B.1 (Bottom), B.2 (Bottom) and B.3 (Bottom)), the circles show very similar results, i.e., the colors do not vary much. In comparison, this does not hold for GRINGO (and IDLV), which suggests that NEWGROUND does in fact not suffer as much from increasing instances density compared to GRINGO and IDLV (Figs. 16 (Top), B.1 (Top), B.2 (Top) and B.3 (Top)). Further, we identify that the rows are also more similar, which is why we overall can confirm hypothesis H1.

Grounding performance. To evaluate the actual grounded programs obtained by the tools, we use our crafted instances for scenarios S1–S4 and generate cactus and scatter plots for each tool to further assess the total value distribution. Note that the plots display either NEWGROUND or NEWGROUND*, depending on which performed better.

Fig. 17 shows the cactus plot of grounding times for scenarios S1–S4 for all grounders. It can be identified that NEWGROUND performs best (in terms of grounding time) in these scenarios with GRINGO performing second-best, except for S4 (nprc). We believe that this is most likely due to the fact that S4 allows to utilize treewidth-based methods (as IDLV uses them). Consequently, the plots show NEWGROUND* for S4, demonstrating that NEWGROUND works well together with IDLV. Similar, Fig. 18 shows grounding times for S5 (stable marriage), where the trend that NEWGROUND performs well continues.

The scatter plots in Fig. 19 (Left) and Fig. 20 (Left) for grounding times depict similar results. Again, we can identify that NEWGROUND improves grounding times in most cases as we see more blue/green dots above the diagonal (the diagonal indicates an equal grounding time of NEWGROUND and GRINGO/IDLV, i.e., points above the diagonal mean a faster grounding time for NEWGROUND).

⁸ For further grounding profiles, see Appendix B.

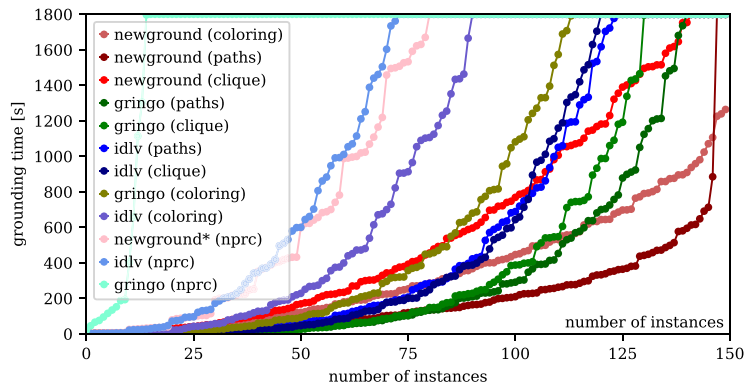


Fig. 17. Cactus plot of grounding time over Scenarios S1–S4 for NEWGROUND (red color tones), GRINGO (green tones), and IDLV (blue tones). The x-axis shows the number of instances and the y-axis the runtime in seconds, sorted in ascending order for each solver individually. The legend is sorted from best (newground (coloring)) to worst (gringo (nprc)). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

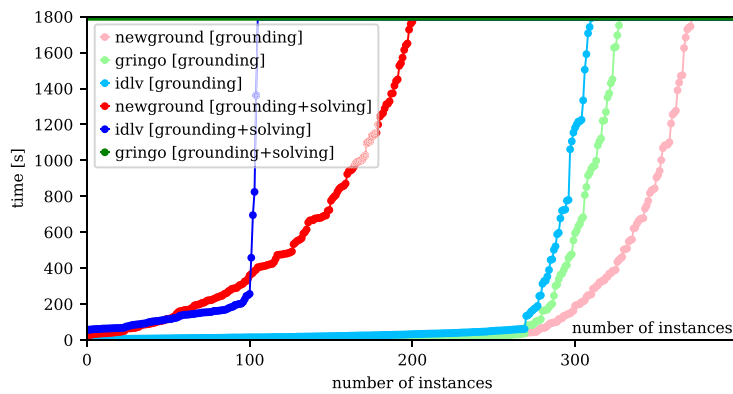


Fig. 18. Cactus plot of grounding time as well as overall performance for Scenario S5 (stable marriage).

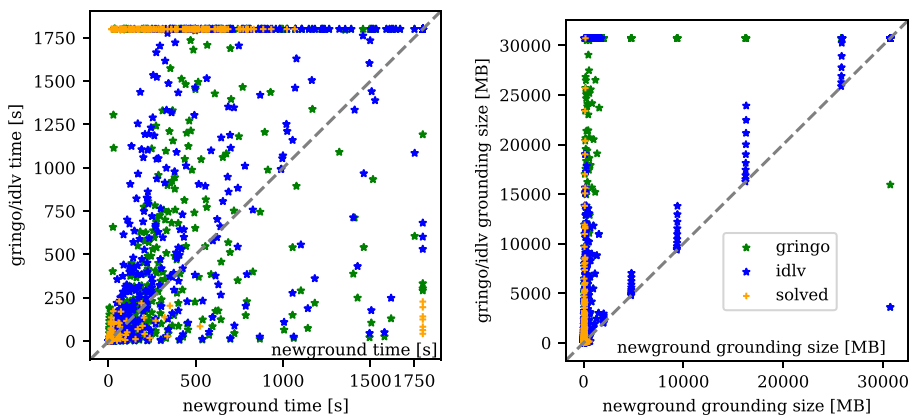


Fig. 19. (Left): Scatter plot of grounding time over Scenarios S1–S4 of NEWGROUND (x-axis) compared to GRINGO and IDLV (y-axis). Green asterisks compare newground with GRINGO and blue asterisks compare newground with IDLV. Overall times (grounding and solving time) of solved instances are highlighted in orange (y-axis shows best result among GRINGO and IDLV). (Right): Scatter plot of grounding size over Scenarios S1–S4 of NEWGROUND (x-axis) compared to both GRINGO (blue) and IDLV (green) on the y-axis. Solved instances are highlighted in orange. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

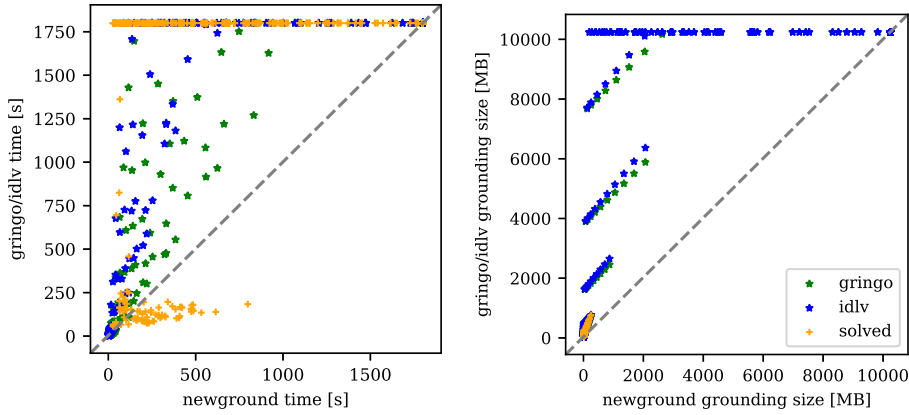


Fig. 20. (Left): Scatter plot of grounding time over S5. Overall times (grounding and solving time) of solved instances are highlighted in orange. (Right): Scatter plot of grounding size over Scenario S5. Those instances that could be solved are highlighted in orange.

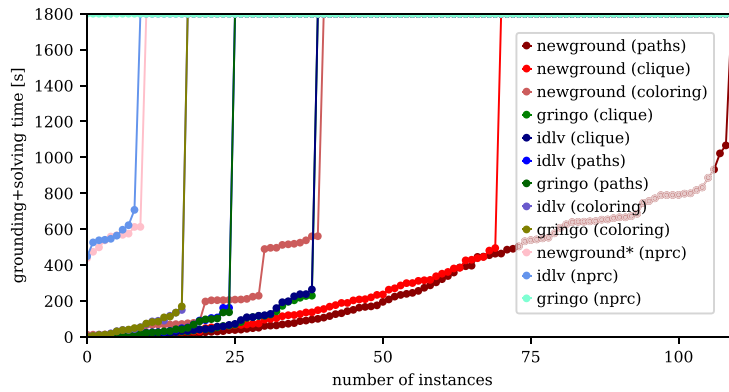


Fig. 21. Corresponding cactus plot of overall (grounding and solving) time over Scenarios S1–S4 (cf. grounding performance of Fig. 17).

Among those below the diagonal (where GRINGO and IDLV are faster), most dots are below 250s, suggesting a potential for a portfolio that uses GRINGO or IDLV for a limited time of 250s and switches to NEWGROUND if unsuccessful. Especially for S5, none of the instances are actually solved beyond that time (orange dots below the diagonal).

The scatter plots of Figs. 19 (Right) and 20 (Right) give some indication regarding grounding sizes. As most of the dots are above the diagonal, it can be seen that NEWGROUND massively reduces grounding sizes. Moreover, there are instances where GRINGO and IDLV output ground programs of size beyond 30GB which are still solved by NEWGROUND.

From these evaluations we can confirm hypothesis H2.

Overall performance. For the evaluation of solvability of the yielded ground programs we refer to Figs. 18 and 21, which show the overall runtime over Scenarios S1–S5. Besides the fact that NEWGROUND still performs better than its contestants, one can identify an immense difference between grounding and solving performance when comparing with Fig. 17. Interestingly, while it may seem unfair to analyze combined grounding and solving time (since NEWGROUND grounds faster), the comparison with the cactus plot for grounding time only reveals that many instances that are grounded within short time by GRINGO or IDLV, but not solved within the remaining 1600s, e.g., many instances of S2 and S3 are grounded by GRINGO within 200s (cf. Fig. 17), but only a fraction of those are solved as Fig. 17 shows. This is especially visible when combining grounding and overall time in a plot, as in Fig. 18 for Scenario S5. While IDLV solves a good amount of grounded instances before dropping out, NEWGROUND can withstand a higher number of instances with a more gentle increase of overall time.

Overall, we see H3 confirmed. Further, throughout the benchmarks we see NEWGROUND operating well with GRINGO and IDLV, which is why we see H4 confirmed as well.

7. Discussion, conclusion, and future work

This work deals with the question whether non-ground logic programs can be efficiently grounded by a translation procedure while circumventing the ASP grounding bottleneck. To that end, we introduce the technique of body-decoupled grounding, where

grounding-intense ASP rules are treated by means of a novel reduction. Relying on the complexity studies for non-ground, normal and tight programs under bounded predicate arity, we first show a reduction to ground, disjunctive programs as a potential alternative to traditional grounding. To do so, the translation replaces non-ground rules by wisely encoding guessing of candidates, saturation and foundedness (and orderings in the case of non-tightness), which allows the decoupling of rule bodies. By decoupling dependencies between body predicates the instantiation can be limited to the linear size of the ground atoms. Next, we partly use the introduced procedure for non-ground, normal programs to design a simpler rule set for the special case of Horn programs. The introduced reductions can be used to check for consistency and atom membership in Horn programs while omitting the expensive encoding of saturation. Lastly, we introduce a reduction for non-ground, disjunctive logic programs in a similar fashion. However, due to the increased complexity, we make use of epistemic logic programs as the target of our reduction. While the core idea of the translation is similar, the encoding involves subset minimization by guessing epistemic atoms and checking saturation for both, potential model and counter witnesses. Nevertheless, body-decoupling is guaranteed through the encoding. While both reductions yield ground programs that are linear in the size of the domain under the setting of bounded predicate arity, without this restriction we still achieve ground programs exponential in the predicate arity.

In terms of runtime, we have shown polynomial runtimes throughout all introduced reduction procedures $\mathcal{R}$, $\mathcal{R}''$, $\mathcal{R}_1$, $\mathcal{R}_F$, and $\mathcal{R}$. We do not expect a significant runtime improvement in the worst case. However, we have seen that orderings tremendously increase the runtime of procedure $\mathcal{R}''$ compared to $\mathcal{R}$ and $\mathcal{R}$, where they are either not needed or can be omitted.

The prototype NEWGROUND implements our optimized reduction procedure $\mathcal{R}'$ for non-ground, tight logic programs. Users can use NEWGROUND by simply providing (tight) logic programs. Besides complete program translations, NEWGROUND supports partially reducibility, such that rules can be explicitly excluded from the reduction. The general idea of the tool is to forward the output of NEWGROUND directly into user-chosen solver to compute any answer sets.

To test our prototype, we ran five benchmark sets comparing against state-of-the-art grounders GRINGO and IDLV. For comparison, we measure grounding sizes, grounding times and solving capabilities of the ground programs. These preliminary experiments indicate that our body-decoupled grounding approach brings advantages throughout most of the benchmarks. However, we are aware of the strengths that sophisticated grounding systems bring, which is why we focused on partial reductions for rules with high body-density. So the results of our benchmarks do not imply the overall advantage of body-decoupled grounding, instead they suggest a well-working interaction between a reduction-approach and a traditional ground-and-solve system. This makes NEWGROUND a prime example for potentially incorporating into any traditional, state-of-the-art grounding system. For this reason, we plan on integrating our approach into (intelligent) grounders, aiming for heuristics that automatically estimate program parts where NEWGROUND likely pays off. In this spirit, recent work advances the idea of body-decoupled grounding via hybrid grounders, which allows the combination with traditional grounders (in the spirit of [Corollary 4.1](#)) [109].

CRediT authorship contribution statement

Viktor Besin: Writing – review & editing, Writing – original draft, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Markus Hecher:** Writing – review & editing, Writing – original draft, Supervision, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Matthias König:** Writing – review & editing, Writing – original draft, Project administration; **Stefan Woltran:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Methodology, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research has been supported by the [Vienna Science and Technology Fund](#) (WWTF) through project [ICT19-065](#), and by the [Austrian Science Fund](#) (FWF) through projects [10.557766/J4656](#), [10.557766/COE12](#), and [P32380](#). Further, Hecher was supported by the French National Research Agency (ANR), through grant ANR-25-CE23-7647-01. Part of the research was carried out while Hecher was a postdoc at MIT, Massachusetts, and while visiting the Simons Institute for the theory of computing at UC Berkeley, California. We thank all the anonymous reviewers for their constructive feedback and many suggestions on how to improve the final version of this work.

Data availability

The code and test instances are available on github via the following link (also provided in the paper): <https://github.com/viktorbesin/newground>.

Appendix A. Additional proofs

A.1. Proof of Theorem 5.5

Theorem 38 (Correctness). Let Π be any disjunctive, non-ground program. Then, the grounding procedure R on Π is correct, i.e., the world views of $R(\Pi)$ restricted to $\text{at}(\mathcal{G}(\Pi))$ match with the answer sets of $\mathcal{G}(\Pi)$. Precisely, (a) for every world view W of $R(\Pi)$, there is an answer set M of $\mathcal{G}(\Pi)$, such that $M = W \cap \text{at}(\mathcal{G}(\Pi))$. Further, (b) every answer set M of $\mathcal{G}(\Pi)$ can be extended to a world view of $R(\Pi)$.

Proof. Claim (a): Let W be a WV of $R(\Pi)$. Then, by Rules (R8), we require that $\text{sat} \in W$. From this, we construct set $M := \{a \mid a \in W \cap \text{at}(\mathcal{G}(\Pi))\}$. Then, it remains to show that (i) M is indeed a model of $\mathcal{G}(\Pi)$ and (ii) that M is a subset-minimal model of $\mathcal{G}(\Pi)^M$, i.e., there does not exist a model N of $\mathcal{G}(\Pi)^M$ with $N \subsetneq M$.

For establishing (i), we assume towards a contradiction that there is a rule $r \in \Pi$ with a concrete ground instantiation $r' \in \mathcal{G}(\{r\})$ such that M is not a model of r' . Then, $B_r^+ \subseteq M$ and $(B_r^- \cup H) \cap M = \emptyset$.

As a consequence, by construction there exists an answer set N of $R(\Pi)^W$ with $\{\text{gr}_x(d) \mid p(\langle x_1, \dots, x_o, x, x_{o+1}, \dots, x_u \rangle) \in B_r^+ \cup B_r^- \cup H_r, p(\langle d_1, \dots, d_o, d, d_{o+1}, \dots, d_u \rangle) \in \text{at}(r')\} \subseteq N$, $\text{sat}_r \notin N$ by Rules (S2) and (S3).

Consequently, by Rules (S4), $\text{sat} \notin N$, which immediately contradicts our assumption that W is a WV due to Rules (R8); hence r' cannot exist.

For showing (ii), we assume towards a contradiction that there exists an interpretation $N \subsetneq M$ that is a $\subseteq$ -smaller model of $\mathcal{G}(\Pi)^M$.

From this we will now argue that therefore W cannot be a WV of $R(\Pi)$. To this end, we construct an interpretation $M' := M \cup (\{h(D) \mid h(X) \in \text{heads}(\Pi), D \in \text{dom}(X)\} \setminus M) \cup \{\text{gr}_x(d), \text{sat}_r, \text{sat} \mid r \in \Pi, x \in \text{var}(r), d \in \text{dom}(x)\}$ as well as interpretations $N' := \{a^c \mid a \in N\} \cup (\{h^c(D) \mid h(X) \in \text{heads}(\Pi), D \in \text{dom}(X)\} \setminus N) \cup \{\text{gr}_x^c(d), \text{sat}_r^c, \text{sat}^c \mid r \in \Pi, x \in \text{var}(r), d \in \text{dom}(x)\}$, and $E := \{\text{nempty} \mid M \neq \emptyset\} \cup \{\text{eq}_a \mid a \in (M \cap N) \cup \{b \mid b \in M'\}\}$.

Then, it remains to show that indeed $M' \cup N' \cup E$ is an answer set of $\mathcal{G}(\Pi)^W$, which, if W was a WV, is impossible by Rules (R9) and due to $\text{sat}^c \in N'$.

Due to W and the construction of M' , Rules (R1)

are facts or not present in $R(\Pi)^W$.

However, since M is a model of $\mathcal{G}(\Pi)$, M' is by construction also a model of Rules (S1)–(S11).

Then, by N , we similarly have that N' is model of Rules (S1)–(S5), as well.

Further, $M' \cup N' \cup E$ is also a model of Rules (R3) (due to N') and of Rules (R4)–(R7) (due to E).

Consequently, $M' \cup N' \cup E$ is also a $\subseteq$ -minimal model of $\mathcal{G}(\Pi)^W$; by construction and similarly to (i), if there was $\subseteq$ -smaller interpretation satisfying Rules (S6)–(S11) without containing some sat_r or sat_r^c , then M or N would not be a model of $\mathcal{G}(\Pi)^W$, respectively.

Observe that due to incomparability of atoms a^c and a^c , there cannot be a smaller model through Rules (R3) as well.

We conclude that, since $M' \cup N' \cup E$ is an answer set of $R(\Pi)^W$ despite $\text{sat}^c \in N'$, and due to Rule (R9), W cannot be a WV of $R(\Pi)$, contradicting our assumption.

Claim (b): Let M be an answer set of $\mathcal{G}(\Pi)$.

First, we construct $N := \{\neg a, a^c, \neg a^c, a \mid h(X) \in \text{heads}(\Pi), D \in \text{dom}(X), a = h(D), a \notin M\}$, which collects those head atoms that are not in M .

From this we construct a set $I := M \cup N \cup \{\neg \text{sat}^c\} \cup \{\text{sat}, \text{sat}_r, \text{gr}_x(d) \mid r \in \Pi, x \in \text{var}(r), d \in \text{dom}(x)\} \cup \{\text{nempty} \mid M \neq \emptyset\} \cup \{\text{eq}_a \mid a \in \text{at}(\mathcal{G}(\Pi)) \setminus M\}$, where we show that $I' \supseteq I$ is a WV of $R(\Pi)$.

Then, it remains to show that I can be extended by some subset S' of $S := \{\text{sat}_r^c, \neg \text{sat}_r^c \mid r \in \Pi\}$ to a WV of $P = R(\Pi)$, i.e., $I' := I \cup S'$ is compatible with $AS(P')$.

Condition (1): $AS(P') \neq \emptyset$ is satisfied as Rules (R8) and (R9) resolve to false constraints by the construction of the epistemic reduct and Rules (R7) only constrain non-empty answer set candidates that yield identical sets of atoms a and a^c . However, these are only some candidates, as Rules (R3) suggest.

Condition (2): for each atom $a \in I$ and for any arbitrary $J \in AS(P')$, we show $a \in J$ as follows. For every atom $a \in M$ the atom $a \in J$ holds (due to a fact in Rules (R1)) in every answer set of P^I by the construction of P^I . Similarly, for every atom $a \in N$, P^I yields a in every answer set (by Rules (R1)). Notice that by construction of N there is no atom a such that $a \in M$ and $a \in N$.

Further, for $\text{nempty} \in I$, we have $\text{nempty} \in J$ since by construction there is some $a \in J$, required by some body of Rules (R6). For any atom of the form $\text{eq}_a \in I$, similarly, by construction of I and Rules (R5), it holds that $\text{eq}_a \in J$. Finally, for the remaining atoms $a \in \{\text{sat}, \text{sat}_r, \text{gr}_x(d) \mid r \in \Pi, x \in \text{var}(r), d \in \text{dom}(x)\}$, we assume towards a contradiction that some $J \in AS(P')$ does not contain a .

Then, however, we have by construction of Rules (S4) and (S5) that there exists a rule $r \in \Pi$ such that $\text{sat}_r \notin J$. In consequence, by Rules (S2) and (S3), we follow that M cannot be a model of $\mathcal{G}(\{r\})$, which contradicts the assumption that M is an answer set of $\mathcal{G}(\Pi)$.

Condition (3): for the negative literals in N , it is easy to see that by construction, none of these can appear in any answer sets $J \in AS(P')$. Further, for the negative literal $\neg \text{sat}^c \in I$, we also have that for every $J \in AS(P')$, $\text{sat}^c \notin J$, which is satisfied as follows. Assume towards a contradiction that $\text{sat}^c \in J$. Then, by construction, $M \neq \emptyset$. Consequently, since J has to be a model of Rules (R7), we can construct a set $N' \subsetneq M$ that is a model of $\mathcal{G}(\Pi)^M$ as follows: $N' := \{a \mid a^c \in J\}$. Since J is an answer set of P^I there cannot be a smaller model $J' \subsetneq J$ with $\text{sat}^c \notin J'$. Therefore, N' is indeed a model of $\mathcal{G}(\Pi)^M$, since, independent of the instantiation

constructed by Rules (S1), every rule $r \in \Pi$ is satisfied by N' due to $\text{sat}^c \in J$ and construction of Rules (S7)–(S9). This, however, contradicts that M is an answer set.

Condition (4): For every atom $b \in \text{at}(P)$ with $\{b, \neg b\} \cap I = \emptyset$, there are $J, J' \in AS(P')$ with $b \in J$ and $b \notin J'$. For each atom of the form $b = a$, one can easily find both J, J' by Rules (R3); similarly, for $b = \text{eq}_a$ we can find J, J' according to Rules (R4) by first setting either $a^c \in J$ or not. Further, by Rules (S6), for atoms of the form $b = \text{gr}_x(d)^c$, one can also easily construct both J, J' accordingly.

Finally, observe that the remaining literals $\ell \in S$ either belong to Condition (1), (2), or (3), where we require either atom $\text{at}(\ell) \in S'$, literal $\neg \text{at}(\ell) \in S'$, or neither, respectively.

This concludes that $I' = I \cup S'$ is a WV of $R(\Pi)$. $\square$

Appendix B. Additional grounding profiles

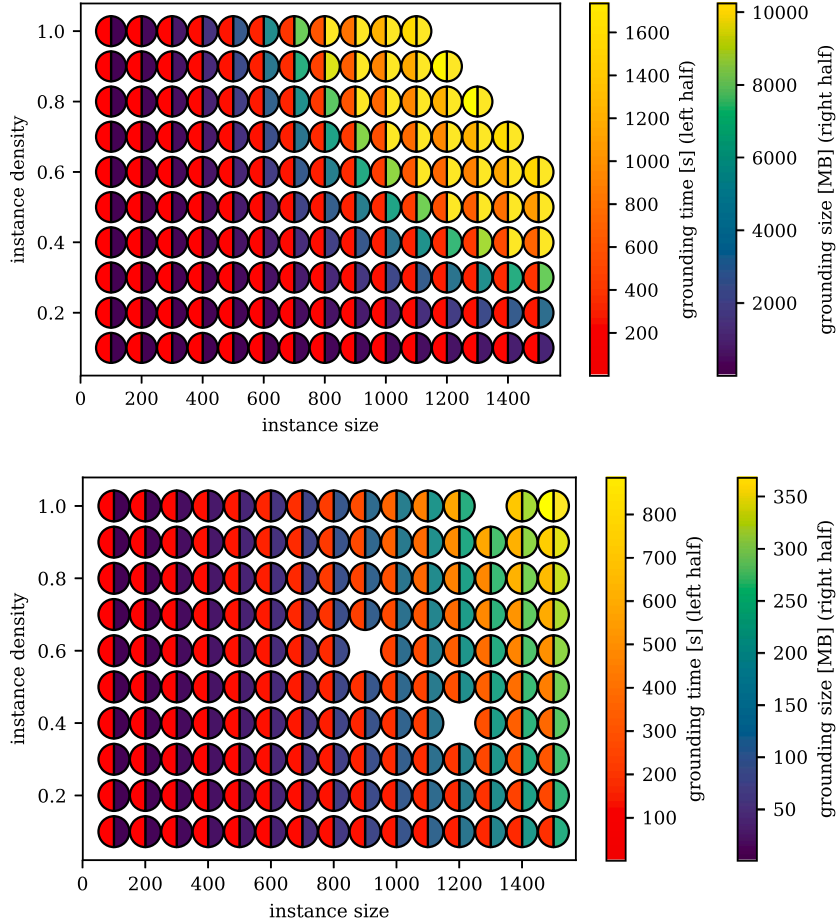


Fig. B.1. (Top): Grounding profile for GRINGO of S2 (paths).
(Bottom): Grounding profile for NEWGROUND of S2.

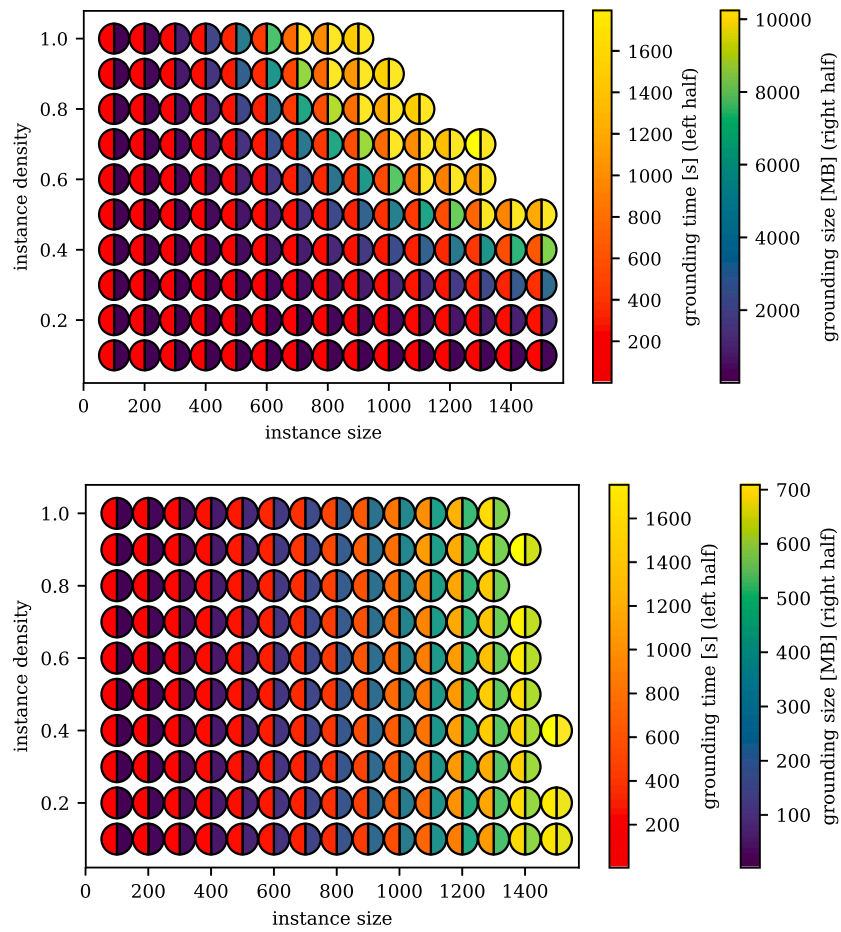


Fig. B.2. (Top): Grounding profile for GRINGO of S3 (clique).
 (Bottom): Grounding profile for NEWGROUND of S3.

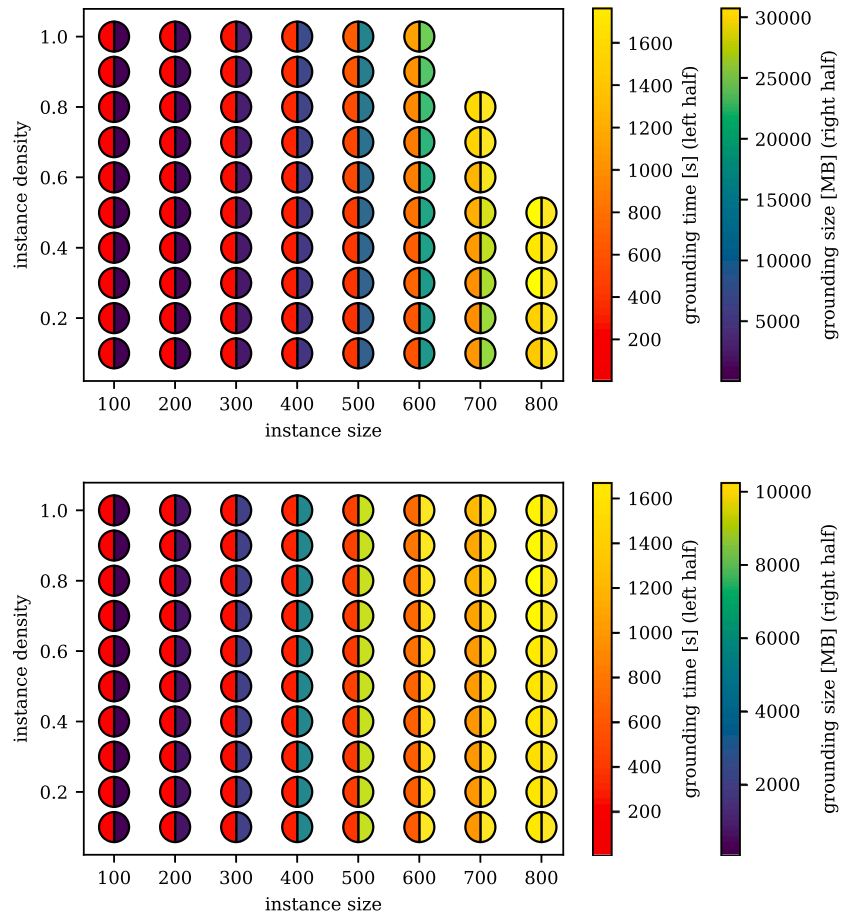


Fig. B.3. (Top): Grounding profile for IDLV of S4 (nprc). (Bottom): Grounding profile for NEWGROUND of S4.

References

- [1] G. Brewka, T. Eiter, M. Truszczyński, Answer set programming at a glance, *Comm. ACM* 54 (12) (2011) 92–103.
- [2] M. Gebser, R. Kaminski, B. Kaufmann, T. Schaub, Multi-shot ASP solving with clingo, *Theory Pract. Log. Program.* 19 (1) (2019) 27–82.
- [3] T. Janhunen, I. Niemelä, The answer set programming paradigm, *AI Mag.* 37 (3) (2016) 13–24.
- [4] T. Schaub, S. Woltran, Special issue on answer set programming, *KI - Kuenstliche Intell.* 32 (2–3) (2018) 101–103.
- [5] A. Biere, M. Heule, H. van Maaren, T. Walsh (Eds.), *Handbook of Satisfiability*, 185 of *Frontiers in Artificial Intelligence and Applications*, IOS Press, 2009.
- [6] H. Kleine Büning, T. Lettmann, *Propositional Logic: Deduction and Algorithms*, 48 of *Cambridge tracts in theoretical computer science*, Cambridge University Press, 1999.
- [7] C.P. Gomes, A. Sabharwal, B. Selman, Model counting, in: *Handbook of Satisfiability - Second Edition*, 336 of *Frontiers in Artificial Intelligence and Applications*, IOS Press, 2021, pp. 993–1014.
- [8] F. Calimeri, S. Perri, J. Zangari, Optimizing answer set computation via heuristic-based decomposition, *Theory Pract. Log. Program.* 19 (4) (2019) 603–628.
- [9] B. Cuteri, C. Dodaro, F. Ricca, P. Schüller, Overcoming the grounding bottleneck due to constraints in ASP solving: constraints become propagators, in: *Proceedings of IJCAI 2020*, ijcai.org, 2020, pp. 1688–1694.
- [10] A. Falkner, G. Friedrich, K. Schekotihin, R. Taupe, E.C. Teppan, Industrial applications of answer set programming, *Künstliche Intell.* 32 (2–3) (2018) 165–176.
- [11] M. Gelfond, Strong introspection, in: *Proceedings of AAAI 1991*, AAAI Press / The MIT Press, 1991, pp. 386–391.
- [12] M. Truszczyński, Revisiting epistemic specifications, in: *Logic Programming, Knowledge Representation, and Nonmonotonic Reasoning - Essays Dedicated to Michael Gelfond on the Occasion of His 65th Birthday*, 6565 of *LNCS*, Springer, 2011, pp. 315–333.
- [13] Y. Shen, T. Eiter, Evaluating epistemic negation in answer set programming, *Artif. Intell.* 237 (2016) 115–135.
- [14] T. Eiter, J.K. Fichte, M. Hecher, S. Woltran, Epistemic logic programs: non-ground and counting complexity, in: *Proceedings of IJCAI 2024*, 2024, pp. 3333–3341.
- [15] B. Kaufmann, N. Leone, S. Perri, T. Schaub, Grounding and solving in answer set programming, *AI Mag.* 37 (3) (2016) 25–32.
- [16] M. Helmert, The fast downward planning system, *J. Artif. Intell. Res.* 26 (2006) 191–246.
- [17] M. Helmert, Concise finite-domain representations for PDDL planning tasks, *Artif. Intell.* 173 (5–6) (2009) 503–535.
- [18] D.Z. Chen, S. Thiébaux, F.W. Trevizan, Learning domain-independent heuristics for grounded and lifted planning, in: M.J. Wooldridge, J.G. Dy, S. Natarajan (Eds.), *Proceedings of AAAI 2024*, AAAI Press, 2024, pp. 20078–20086.
- [19] A.B. Corrêa, G. De Giacomo, Lifted planning: recent advances in planning using first-order representations, in: *Proceedings of IJCAI 2024*, ijcai.org, 2024, pp. 8010–8019.
- [20] A.B. Corrêa, M. Hecher, M. Helmert, D.M. Longo, F. Pommerening, S. Woltran, Grounding planning tasks using tree decompositions and iterated solving, in: *Proceedings of ICAPS 2023*, AAAI Press, 2023, pp. 100–108.
- [21] M. Bannach, M. Hecher, Counting solutions under cardinality constraints: structure counts in counting, in: *Proceedings of KR 2025*, 2025. In Press.
- [22] T. Eiter, G. Gottlob, On the computational cost of disjunctive logic programming: propositional case, *Ann. Math. Artif. Intell.* 15 (3–4) (1995) 289–323.
- [23] N. Bidoit, C. Froidevaux, Negation by default and unstratifiable logic programs, *Theor. Comput. Sci.* 78 (1) (1991) 85–112.
- [24] W. Marek, M. Truszczyński, Autoepistemic logic, *J. ACM* 38 (3) (1991) 588–619.
- [25] E. Dantsin, T. Eiter, G. Gottlob, A. Voronkov, Complexity and expressive power of logic programming, *ACM Comput. Surv.* 33 (3) (2001) 374–425.
- [26] T. Eiter, W. Faber, M. Fink, S. Woltran, Complexity results for answer set programming with bounded predicate arities and implications, *Ann. Math. Artif. Intell.* 51 (2–4) (2007) 123–165.
- [27] P. Cabalar, J. Pandinno, J. Gareia, J. Romero, T. Schaub, eclingo : a solver for epistemic logic programs, *Theory Pract. Log. Program.* 20 (6) (2020) 834–847.
- [28] M. Bichler, M. Morak, S. Woltran, selp: a single-shot epistemic logic program solver, *Theory Pract. Log. Program.* 20 (4) (2020) 435–455.
- [29] V. Besin, M. Hecher, S. Woltran, Utilizing treewidth for quantitative reasoning on epistemic logic programs, *Theory Pract. Log. Program.* 21 (5) (2021) 575–592.
- [30] V. Besin, M. Hecher, S. Woltran, Body-decoupled grounding via solving: a novel approach on the ASP bottleneck, in: *Proceedings of IJCAI 2022*, ijcai.org, 2022, pp. 2546–2552.
- [31] V. Besin, M. Hecher, S. Woltran, On the structural complexity of grounding - tackling the ASP grounding bottleneck via epistemic programs and treewidth, in: *Proceedings ECAI 2023*, IOS Press, 2023, pp. 247–254.
- [32] M. Gelfond, N. Leone, Logic programming and knowledge representation—the A-prolog perspective, *Artif. Intell.* 138 (1) (2002) 3–38.
- [33] M. Gelfond, V. Lifschitz, The stable model semantics for logic programming, in: *Proceedings of JICSLP88*, 1988, pp. 1070–1080.
- [34] M. Gelfond, V. Lifschitz, Classical negation in logic programs and disjunctive databases, *New Gener. Comput.* 9 (3/4) (1991) 365–386.
- [35] D. Pearce, A new logical characterisation of stable models and answer sets, in: *Proceedings of NMELP96*, 1216 of *Lecture Notes in Computer Science*, 1996, pp. 57–70.
- [36] R. Kaminski, J. Romero, T. Schaub, P. Wanko, How to build your own ASP-based system?!, *Theory Pract. Log. Program.* 23 (1) (2023) 299–361.
- [37] T. Eiter, G. Ianni, T. Krennwallner, Answer set programming: a primer, in: *Reasoning Web*, 5689 of *Lecture Notes in Computer Science*, 2009, pp. 40–110.
- [38] P. Simons, I. Niemelä, T. Soeninen, Extending and implementing the stable model semantics, *Artif. Intell.* 138 (1–2) (2002) 181–234.
- [39] N. Leone, G. Pfeifer, W. Faber, T. Eiter, G. Gottlob, S. Perri, F. Scarcello, The DLV system for knowledge representation and reasoning, *ACM Trans. Comput. Log.* 7 (3) (2006) 499–562.
- [40] Y. Lierler, M. Maratea, Cmodels-2: SAT-based answer set solver enhanced to non-tight programs, in: *Proceedings of LPNMR03*, 2923 of *Lecture Notes in Computer Science*, 2003, pp. 346–350.
- [41] F. Lin, Y. Zhao, ASSAT: computing answer sets of a logic program by SAT solvers, *Artif. Intell.* 157 (1–2) (2004) 115–137.
- [42] T. Janhunen, Implementing stable-unstable semantics with ASPTOOLS and clingo, in: *Proceedings of PADL22*, 13165 of *Lecture Notes in Computer Science*, 2022, pp. 135–153.
- [43] M. Gebser, B. Kaufmann, T. Schaub, Conflict-driven answer set solving: from theory to practice, *Artif. Intell.* 187–188 (2012) 52–89.
- [44] M. Gebser, R. Kaminski, B. Kaufmann, J. Romero, T. Schaub, Progress in clasp Series 3, in: *Proceedings of LPNMR 2015*, 2015, pp. 368–383.
- [45] M. Alviano, C. Dodaro, N. Leone, F. Ricca, Advances in WASP, in: *Proceedings of LPNMR 2015*, 9345 of *Lecture Notes in Computer Science*, 2015, pp. 40–54.
- [46] R. Kaminski, T. Schaub, On the foundations of grounding in answer set programming, *Theory Pract. Log. Program.* 23 (6) (2023) 1138–1197.
- [47] T. Syrjänen, Omega-restricted logic programs, in: *Proceedings of LPNMR01*, 2173 of *Lecture Notes in Computer Science*, 2001, pp. 267–280.
- [48] M. Gebser, T. Schaub, S. Thiele, Gringo: a new grounder for answer set programming, in: *Proceedings of LPNMR07*, 4483 of *Lecture Notes in Computer Science*, 2007, pp. 266–271.
- [49] M. Gebser, A. Harrison, R. Kaminski, V. Lifschitz, T. Schaub, Abstract gringo, *Theory Pract. Log. Program.* 15 (4) (2015) 449–463.
- [50] W. Faber, N. Leone, S. Perri, The intelligent grounder of DLV, in: *Correct Reasoning - Essays on Logic-Based AI in Honour of Vladimir Lifschitz*, 7265 of *Lecture Notes in Computer Science*, 2012, pp. 247–264.
- [51] F. Calimeri, D. Fuscà, S. Perri, J. Zangari, I-DLV: the new intelligent grounder of DLV, *Intell. Artif.* 11 (1) (2017) 5–20.
- [52] M. Gebser, R. Kaminski, B. Kaufmann, M. Ostrowski, T. Schaub, P. Wanko, Theory solving made easy with Clingo 5, in: *Proceedings of ICLP16*, 2016, pp. 1–15.
- [53] T. Eiter, J.K. Fichte, M. Hecher, S. Woltran, Epistemic logic programs: non-ground and counting complexity, in: *33rd International Joint Conference on Artificial Intelligence (IJCAI)*, ijcai.org, 2024, pp. 3333–3341.
- [54] M. Gebser, N. Leone, M. Maratea, S. Perri, F. Ricca, T. Schaub, Evaluation techniques and systems for answer set programming: a survey, in: *Proceedings of IJCAI 2018*, ijcai.org, 2018, pp. 5450–5456.
- [55] E. Tsamoura, V. Gutiérrez-Basulto, A. Kimmig, Beyond the grounding bottleneck: datalog techniques for inference in probabilistic logic programs, in: *AAAI'20*, AAAI Press, 2020, pp. 10284–10291.
- [56] M. Alviano, G. Amendola, C. Dodaro, N. Leone, M. Maratea, F. Ricca, Evaluation of disjunctive programs in WASP, in: *Proceedings of LPNMR 2019*, 11481 of *LNCS*, Springer, 2019, pp. 241–255.

- [57] N. Hippen, Y. Lierler, Estimating grounding sizes of logic programs under answer set semantics, in: *Proceedings of JELIA 2021*, 12678 of *LNCS*, Springer, 2021, pp. 346–361.
- [58] J. Bomanson, T. Janhunen, A. Weinzierl, Enhancing lazy grounding with lazy normalization in answer-set programming, in: *Proceedings of AAAI 2019*, AAAI Press, 2019, pp. 2694–2702.
- [59] A. Weinzierl, R. Taupe, G. Friedrich, Advancing lazy-grounding ASP solving techniques - restarts, phase saving, heuristics, and more, *Theory Pract. Log. Program.* 20 (5) (2020) 609–624.
- [60] M. Banbara, B. Kaufmann, M. Ostrowski, T. Schaub, Clingcon: the next generation, *Theory Pract. Log. Program.* 17 (4) (2017) 408–461.
- [61] T. Janhunen, R. Kaminski, M. Ostrowski, S. Schellhorn, P. Wanko, T. Schaub, Clingo goes linear constraints over reals and integers, *Theory Pract. Log. Program.* 17 (5–6) (2017) 872–888.
- [62] P. Cabalar, J. Fandinno, T. Schaub, P. Wanko, A uniform treatment of aggregates and constraints in hybrid ASP, in: *Proceedings of KR 2020*, 2020, pp. 193–202.
- [63] M. Bichler, M. Morak, S. Woltran, Lpopt: a rule optimization tool for answer set programming, *Fundam. Inform.* 177 (3–4) (2020) 275–296.
- [64] B. Bliem, M. Morak, M. Moldovan, S. Woltran, The impact of treewidth on grounding and solving of answer set programs, *J. Artif. Intell. Res.* 67 (2020) 35–80.
- [65] D. Mitchell, Guarded constraint models define treewidth preserving reductions, in: *Proceedings of CP 2019*, 11802 of *LNCS*, Springer, 2019, pp. 350–365.
- [66] A.D. Palù, A. Dovier, E. Pontelli, G. Rossi, GASP: answer set programming with lazy grounding, *Fundam. Inform.* 96 (3) (2009) 297–322.
- [67] C. Lefèvre, P. Nicolas, The first version of a new ASP solver: ASPeRiX, in: *Proceedings of LPNMR09*, 5753 of *Lecture Notes in Computer Science*, 2009, pp. 522–527.
- [68] M. Dao-Tran, T. Eiter, M. Fink, G. Weidinger, A. Weinzierl, OMiGA: an open minded grounding on-the-fly answer set solver, in: *Proceedings of JELIA12*, 7519 of *Lecture Notes in Computer Science*, 2012, pp. 480–483.
- [69] A. Weinzierl, Blending lazy-grounding and CDNL search for answer-set solving, in: *Proceedings of LPNMR 2017*, 2017, pp. 191–204.
- [70] B. De Cat, M. Denecker, M. Bruynooghe, P. Stuckey, Lazy model expansion: interleaving grounding with search, *J. Artif. Intell. Res.* 52 (2015) 235–286.
- [71] L. Leutgeb, A. Weinzierl, Techniques for efficient lazy-grounding ASP solving, in: *Proceedings of DECLARE 2018*, 2018, pp. 132–148.
- [72] B. Cuteri, C. Dodaro, F. Ricca, P. Schüller, Partial compilation of ASP programs, *Theory Pract. Log. Program.* 19 (5) (2019) 857–873.
- [73] Y. Lierler, J. Robbins, DualGrounder: lazy instantiation via Clingo multi-shot framework, in: *Proceedings of JELIA 2021*, 12678, 2021, pp. 435–441.
- [74] B. Cuteri, C. Dodaro, F. Ricca, P. Schüller, Constraints, lazy constraints, or propagators in ASP solving: an empirical analysis, *Theory Pract. Log. Program.* 17 (5) (2017) 780–799.
- [75] C. Dodaro, G. Mazzotta, F. Ricca, Compilation of tight ASP programs, in: *Proceedings of ECAI 2023*, 372 of *Frontiers in Artificial Intelligence and Applications*, IOS Press, 2023, pp. 557–564.
- [76] G. Mazzotta, F. Ricca, C. Dodaro, Compilation of aggregates in ASP systems, in: *Proceedings of AAAI 2022*, AAAI Press, 2022, pp. 5834–5841.
- [77] Y. Dimopoulos, M. Gebser, P. Lühne, J. Romero, T. Schaub, plasp 3: towards effective ASP planning, *Theory Pract. Log. Program.* 19 (3) (2019) 477–504.
- [78] G. Brewka, J.P. Delgrande, J. Romero, T. Schaub, Asprin: customizing answer set preferences without a headache, in: *Proceedings AAAI 2015*, AAAI Press, 2015, pp. 1467–1474.
- [79] W. Faber, S. Woltran, Manifold answer-set programs and their applications, in: *Logic Programming, Knowledge Representation, and Nonmonotonic Reasoning - Essays Dedicated to Michael Gelfond on the Occasion of His 65th Birthday*, 6565 of *LNCS*, Springer, 2011, pp. 44–63.
- [80] T. Eiter, A. Polleres, Towards automated integration of guess and check programs in answer set programming: a meta-interpreter and applications, *Theory Pract. Log. Program.* 6 (1–2) (2006) 23–60.
- [81] M. Ostrowski, T. Schaub, ASP Modulo CSP: the Clingcon system, *Theory Pract. Log. Program.* 12 (4) (2012) 485–503.
- [82] M. Balduccini, Y. Lierler, Constraint answer set solver EZCSP and why integration schemas matter, *Theory Pract. Log. Program.* 17 (4) (2017) 462–515.
- [83] B. Susman, Y. Lierler, System description: SMT-based constraint answer set solver EZSMT, in: *Proceedings of ICLP16*, 2016, pp. 1–15.
- [84] G. Liu, T. Janhunen, I. Niemelä, Answer set programming via mixed integer programming, in: *Proceedings of KR12*, 2012, pp. 32–42.
- [85] T. Eiter, M. Fink, G. Ianni, T. Krennwallner, C. Redl, P. Schüller, A model building framework for answer set programming with external computations, *Theory Pract. Log. Program.* 16 (4) (2016) 418–464.
- [86] R.A. Aziz, G. Chu, P.J. Stuckey, Stable model semantics for founded bounds, *Theory Pract. Log. Program.* 13 (4) (2013) 517–532.
- [87] A. Beiser, M. Hecher, K. Unalan, S. Woltran, Bypassing the ASP bottleneck: hybrid grounding by splitting and rewriting, in: *Proceedings of IJCAI 2024*, ijcai.org, 2024, pp. 3250–3258.
- [88] M. Morak, S. Woltran, Preprocessing of complex non-ground rules in answer set programming, in: *Proceedings of ICLP12*, 2012, pp. 247–258.
- [89] T. Eiter, A. Polleres, Towards automated integration of guess and check programs in answer set programming: a meta-interpreter and applications, in: *Proceedings of LPNMR04*, 2004, pp. 100–113.
- [90] T. Eiter, W. Faber, M. Muskhofa, Space efficient evaluation of ASP programs with bounded predicate arities, in: *Proceedings of AAAI 2010*, AAAI Press, 2010, pp. 303–308.
- [91] F. Fages, Consistency of Clark's completion and existence of stable models, *Log. Methods Comput. Sci.* 1 (1) (1994) 51–60.
- [92] T. Eiter, W. Faber, M. Fink, S. Woltran, Complexity results for answer set programming with bounded predicate arities and implications, *Ann. Math. Artif. Intell.* 51 (2007) 123–165.
- [93] F. Lin, J. Zhao, On tight logic programs and yet another translation from normal logic programs to propositional logic, in: *Proceedings of IJCAI 2003*, Morgan Kaufmann, 2003, pp. 853–858.
- [94] T. Janhunen, Some (in)translatability results for normal logic programs and propositional theories, *J. Appl. Non-Class. Log.* 16 (1–2) (2006) 35–86.
- [95] A.V. Gelder, K.A. Ross, J.S. Schlipf, The well-founded semantics for general logic programs, *J. ACM* 38 (3) (1991) 620–650.
- [96] M. Alviano, F. Calimeri, W. Faber, N. Leone, S. Perri, Unfounded sets and well-founded semantics of answer set programs with aggregates, *J. Artif. Intell. Res.* 42 (2011) 487–527.
- [97] V.W. Marek, V.S. Subrahmanian, The relationship between stable, supported, default and autoepistemic semantics for general logic programs, *Theor. Comput. Sci.* 103 (2) (1992) 365–386.
- [98] U. Egly, S.A. Gaggl, S. Woltran, Answer-set programming encodings for argumentation frameworks, *Argument Comput.* 1 (2) (2010) 147–177.
- [99] G. Amendola, C. Dodaro, W. Faber, F. Ricca, Paracoherent answer set computation, *Artif. Intell.* 299 (2021) 103519.
- [100] U. Egly, T. Eiter, H. Tompits, S. Woltran, Solving advanced reasoning tasks using quantified boolean formulas, in: *Proceedings of AAAI 2000*, AAAI Press / The MIT Press, 2000, pp. 417–422.
- [101] E. Giunchiglia, P. Marin, M. Narizzano, Reasoning with quantified boolean formulas, in: A. Biere, M. Heule, H. van Maaren, T. Walsh (Eds.), *Handbook of Satisfiability*, 185 of *Frontiers in Artificial Intelligence and Applications*, IOS Press, 2009, pp. 761–780.
- [102] M. Gebser, R. Kaminski, B. Kaufmann, T. Schaub, Answer Set Solving in Practice, Morgan & Claypool, 2012.
- [103] M. Truszczyński, Trichotomy and dichotomy results on the complexity of reasoning with disjunctive logic programs, *Theory Pract. Log. Program.* 11 (6) (2011) 881–904.
- [104] P. Kahl, R. Watson, E. Balai, M. Gelfond, Y. Zhang, The language of epistemic specifications (refined) including a prototype solver, *J. Log. Comput.* 30 (4) (2020) 953–989.
- [105] M. Morak, Epistemic logic programs: a different world view, in: *Proc. ICLP*, 2019, pp. 52–64.
- [106] M. Gebser, R. Kaminski, T. Schaub, Complex optimization in answer set programming, *Theory Pract. Log. Program.* 11 (4–5) (2011) 821–839.
- [107] L.G. Valiant, The complexity of enumeration and reliability problems, *SIAM J. on Computing* 8 (3) (1979) 410–421.
- [108] A. Beiser, S. Woltran, M. Hecher, Automated hybrid grounding using structural and data-driven heuristics, *Theory Pract. Log. Program.* 25 (4) (2025) 489–506.
- [109] A. Beiser, M. Hecher, K. Unalan, S. Woltran, Bypassing the ASP bottleneck: hybrid grounding by splitting and rewriting, in: *Proceedings of IJCAI 2024*, 2024, pp. 3250–3258.